SerialPortManager

SerialPortManager Manuale utente

Versione 0.3.0-rc.6 - Edizione 2026-10-11

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

SerialPortManager è un software proprietario di Fation Coga, concesso in licenza per PC (vedi LICENSE.md). Questo manuale fa parte del prodotto: nessuna sua parte può essere riprodotta o distribuita senza il permesso scritto del titolare dei diritti.

Indice

1. Introduzione **5**

1.1 Cosa si può fare . . . 5

1.2 Questo manuale . . . 6

2. Installazione e file **6**

2.1 Requisiti . . . 6

2.2 Installazione . . . 6

2.3 La cartella dati . . . 6

2.4 La cartella manuals . . . 7

2.5 Disinstallazione . . . 7

3. Primo avvio e prova **7**

4. La finestra principale **7**

4.1 Schede (slot) . . . 8

4.2 Menu . . . 9

4.3 Barra di stato . . . 9

4.4 Log dell'applicazione . . . 10

5. Seriale > Porte seriali: quante porte, nomi e impostazioni **10**

5.1 Una porta appartiene a una sola scheda . . . 10

6. Impostazioni della porta **10**

6.1 Apertura e chiusura . . . 11

6.2 Linee di controllo e Break . . . 11

6.3 Porte perse e descrizioni . . . 12

7. La casella Invia **12**

7.1 Notazioni . . . 12

7.2 Tabella Inserisci carattere . . . 13

7.3 Contatore / CRC... . . . 13

7.4 Comandi e cronologia . . . 14

8. La console **14**

8.1 Righe . . . 14

8.2 Opzioni della console . . . 15

8.3 Suggerimento . . . 15

8.4 Opzioni di ricezione: suddivisione e verifica del checksum . . . 16

8.5 Cercare, filtrare ed evidenziare . . . 16

8.6 Tempo tra i messaggi . . . 16

8.7 Statistiche . . . 17

8.8 Cattura, riproduzione ed esportazione . . . 17

9. Risposte automatiche e messaggi periodici **17**

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

9.1 Interruttori ed elenchi . . . 17
9.2 Risposta automatica . . . 18
9.3 Messaggio periodico . . . 18
9.4 L'editor del messaggio . . . 18
9.5 Valori incrementali . . . 19
9.6 Checksum . . . 19
10. Inoltro **20**
11. Trasferimento file **21**
11.1 Inviare un file . . . 21
11.2 Opzioni di invio . . . 21
11.3 Le due modalità di invio . . . 22
11.4 Catturare i file . . . 22
11.5 Opzioni di cattura . . . 23
11.6 Recupero di un trasferimento interrotto . . . 23
12. Clic destro sulla console: analisi di un messaggio **24**
13. L'Analizzatore di byte **25**
13.1 Checksum / CRC . . . 25
13.2 Interi / virgola mobile . . . 27
13.3 Segnale . . . 27
13.4 Protocollo . . . 28
14. La finestra Protocolli **29**
14.1 La scheda Panoramica . . . 29
14.2 Le schede dei protocolli . . . 30
14.3 Parametri del simulatore . . . 30
14.4 La vista della forma d'onda . . . 32
15. Il Master Modbus **33**
16. Il Plotter seriale **34**
17. Sequenze di test **34**
18. Profili **35**
19. Licenze **36**
19.1 Prova, con licenza e senza licenza . . . 36
19.2 La finestra Licenza . . . 36
19.3 Ottenere una licenza . . . 37
19.4 Che cosa controlla una licenza . . . 37
20. Impostazioni **37**
20.1 Tema . . . 37
20.2 Lingua . . . 38

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

20.3 Livello log 38
20.4 Apri cartella dati 38
20.5 Database eventi 38
21. Menu Aiuto **38**
22. Risoluzione dei problemi **39**
23. Scorciatoie da tastiera e mouse **40**
24. Glossario **40**
25. Note legali **41**

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Versione 0.3.0 - SerialPortManager (SerialPM) per Windows.

Copyright (C) 2026 Fation Coga. Tutti i diritti riservati.

1. Introduzione

SerialPortManager (SerialPM) è uno strumento per Windows dedicato alle porte seriali: porte COM del PC, adattatori USB-seriale, convertitori RS-232, RS-422 e RS-485 e qualsiasi dispositivo che si presenta come porta COM. Mostra tutto ciò che viene ricevuto e inviato, invia dati scritti in esadecimale o come testo, risponde automaticamente ai messaggi ricevuti, invia messaggi periodici, inoltra i dati da una porta a un'altra, invia e riceve file interi e analizza i byte: checksum e CRC, numeri interi e in virgola mobile in ogni ordine dei byte, frame di protocollo (Modbus, NMEA) e la forma d'onda che i byte producono sui fili.

Si possono controllare più porte contemporaneamente: ogni porta ha la propria scheda (uno *slot*) con la propria connessione, il proprio thread in background, la propria console, la propria automazione e il proprio trasferimento file.

1.1 Cosa si può fare

- Aprire una o più porte seriali, ognuna con la propria velocità, bit di dati, parità, bit di stop, controllo di flusso e timeout.
- Inviare byte scritti in notazione **Hex**, **ASCII** o **Misto**, con una tabella di tutti i 256 caratteri.
- Leggere una console che mostra ogni messaggio ricevuto (RX) e inviato (TX) con ora, testo e byte esadecimali, in colori diversi secondo il tipo.
- Definire **risposte automatiche**: quando si ricevono certi byte, rispondere con un messaggio, eventualmente dopo un'attesa.
- Definire **messaggi periodici**: inviare un messaggio a intervalli fissi.
- Inserire **valori incrementali** (contatori) e **checksum** (SUM, XOR, LRC, CRC-8/16/32) nelle risposte e nei messaggi periodici.
- **Inoltrare** tutto ciò che una porta riceve a un'altra porta.
- **Inviare un file** a blocchi, con pausa o con conferma blocco per blocco, con prefisso iniziale, marcatore di fine e recupero dei trasferimenti interrotti; **catturare i file** inviati in questo modo.
- Fare clic destro su qualsiasi messaggio della console - o su una selezione dei suoi byte - per checksum, numeri, simulazione del segnale e decodifica del protocollo.
- Aprire l'**Analizzatore di byte** (circa 50 algoritmi di checksum e CRC, qualsiasi CRC dai suoi parametri, ogni tipo intero e in virgola mobile, decodifica Modbus RTU / Modbus ASCII / NMEA 0183, forma d'onda).
- Aprire la finestra **Protocolli**: spiegazioni e simulazioni dal vivo di UART, RS-232, RS-422, RS-485, SPI, I²C, 1-Wire, CAN, LIN, JTAG e delle codifiche di linea (NRZ, Manchester...).
- Impostare le uscite **RTS** e **DTR**, osservare **CTS**, **DSR**, **DCD** e **RI**, inviare un **Break**; far riaprire una porta USB persa quando ritorna.
- Dividere i byte ricevuti in messaggi per **pausa**, **delimitatore**, **lunghezza fissa**, **campo lunghezza** o **temporizzazione Modbus RTU**, e verificare il **checksum / CRC** di ogni messaggio ricevuto.
- **Cercare**, **filtrare** ed **evidenziare** nella console, vedere il **tempo tra i messaggi** e le **statistiche** di una porta.
- **Catturare** il traffico in un file, **riprodurlo**, **esportare** la console come testo o CSV; tenere **comandi salvati** e la **cronologia** di ciò che si è inviato.

- Usare il **Master Modbus**, il **Plotter seriale** e le **Sequenze di test** (script invia / attendi con esito superato / fallito); spostare un'intera configurazione con i **profili**.
- Lavorare in italiano o in inglese, con tema chiaro o scuro.

1.2 Questo manuale

Le etichette dell'interfaccia sono scritte in **grassetto** esattamente come il programma le mostra in italiano. I percorsi dei menu sono scritti come **Menu > Voce**. Le scorciatoie da tastiera sono scritte come Ctrl-B, F1 e così via.

Il sistema di licenze decide quali funzioni sono disponibili: vedi il capitolo 19. Durante i 14 giorni di prova tutte le funzioni sono disponibili.

2. Installazione e file

2.1 Requisiti

- Windows (build a 64 o 32 bit, come fornita).
- Almeno una porta seriale (porta COM, adattatore USB-seriale o porta virtuale) per comunicare con un dispositivo. Senza porte il programma si avvia comunque: l'Analizzatore di byte e la finestra Protocolli funzionano senza alcuna porta.

2.2 Installazione

SerialPortManager non richiede un programma di installazione. Copiare la cartella fornita in una posizione in cui si può scrivere, per esempio una cartella nei propri documenti, e avviare **SerialPortManager.exe**. Non metterla in una cartella in cui Windows non consente la scrittura (per esempio *Programmi*) a meno di avviarla con i permessi di scrittura: il programma conserva i suoi dati accanto all'exe (vedi 2.3).

La cartella fornita contiene:

Elemento	Che cos'è
SerialPortManager.exe	il programma
file DLL	solo nelle build che li usano: librerie wxWidgets, SQLite e serial
SerialPortManager data	la cartella dati (creata al primo avvio se manca)
manuals	i manuali PDF aperti dal menu Aiuto
THIRD-PARTY-NOTICES.md	le licenze dei componenti di terze parti

2.3 La cartella dati

Tutto ciò che il programma scrive va nella cartella **SerialPortManager data** accanto all'exe, così lo stato completo di un'installazione è una sola cartella da salvare, spostare o cancellare. Se quella cartella non si può creare (posizione di sola lettura), viene usata la cartella dell'exe. **Impostazioni > Apri cartella dati** la apre in Esplora risorse e **Aiuto > Informazioni versione...** ne mostra il percorso completo.

File	Contenuto
AppSettings.txt	tutte le impostazioni: porte e loro nomi, testo della casella di invio, risposte automatiche, messaggi periodici, opzioni dei file, opzioni della console, tema, lingua, livello log, parametri del simulatore
events.db	un database SQLite con una riga per evento: avvio e chiusura del programma, stato della licenza, porta aperta / chiusa, ogni messaggio ricevuto e inviato (RX, TX, risposta automatica, periodico, inoltrato) in esadecimale, file inviati e ricevuti
log.txt	il log dell'applicazione (le stesse righe del Log dell'applicazione in fondo alla finestra), accodato a ogni avvio

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

File	Contenuto
language-it.xml	la traduzione italiana (un file per ogni lingua aggiuntiva: language-xx.xml)
license.lic	la licenza installata, se presente
license.lic.bak	la licenza precedente, dopo l'installazione di una nuova o la rimozione
license-request.txt	la richiesta di licenza di questo PC, scritta automaticamente finché il programma non ha licenza
license-state.dat	lo stato della prova (non modificarlo: una copia modificata fa terminare la prova)

Il database degli eventi si può aprire con qualsiasi strumento SQLite; la sua tabella **events** ha le colonne *time_utc*, *kind* (app, license, port, rx, tx, tx-auto, tx-periodic, tx-forward, file) e *text*. I byte ricevuti e inviati sono preceduti dal titolo della loro scheda, per esempio `[COM3 - Contatore] 01 03 00 00 00 01 84 0A`.

2.4 La cartella manuals

Aiuto > Manuale utente (F1) e **Aiuto > Guida rapida** aprono i file PDF della cartella **manuals** accanto all'exe, nella lingua in uso: SerialPM-Manual-IT.pdf / SerialPM-Manual-EN.pdf e SerialPM-QuickGuide-IT.pdf / SerialPM-QuickGuide-EN.pdf. Si aprono nel visualizzatore PDF predefinito di Windows. Se un file manca, il programma lo segnala: *"Il manuale non è stato trovato o non può essere aperto: ... I manuali sono installati nella cartella "manuals" accanto al programma."*

2.5 Disinstallazione

Chiudere il programma e cancellarne la cartella. Per conservare impostazioni e licenza per un'installazione successiva, tenere una copia della cartella **SerialPortManager data**. Lo stato della prova è conservato anche nel registro di Windows dell'utente (HKCU\Software\SerialPortManager\License); cancellare i file non fa ripartire la prova.

3. Primo avvio e prova

Al primo avvio su un PC:

1. Il programma crea la cartella dati e il log.
2. Vengono usati la lingua inglese e il tema chiaro (si cambiano dal menu **Impostazioni**; il menu inglese si chiama *Settings*).
3. Inizia la prova di 14 giorni: tutte le funzioni sono disponibili. La barra di stato mostra **Prova: N giorno/i rimasti**.
4. Nella cartella dati viene scritta una richiesta di licenza (license-request.txt), pronta da inviare all'emittente della licenza.
5. Si apre la finestra principale con una scheda, **Slot 1**, senza porta.

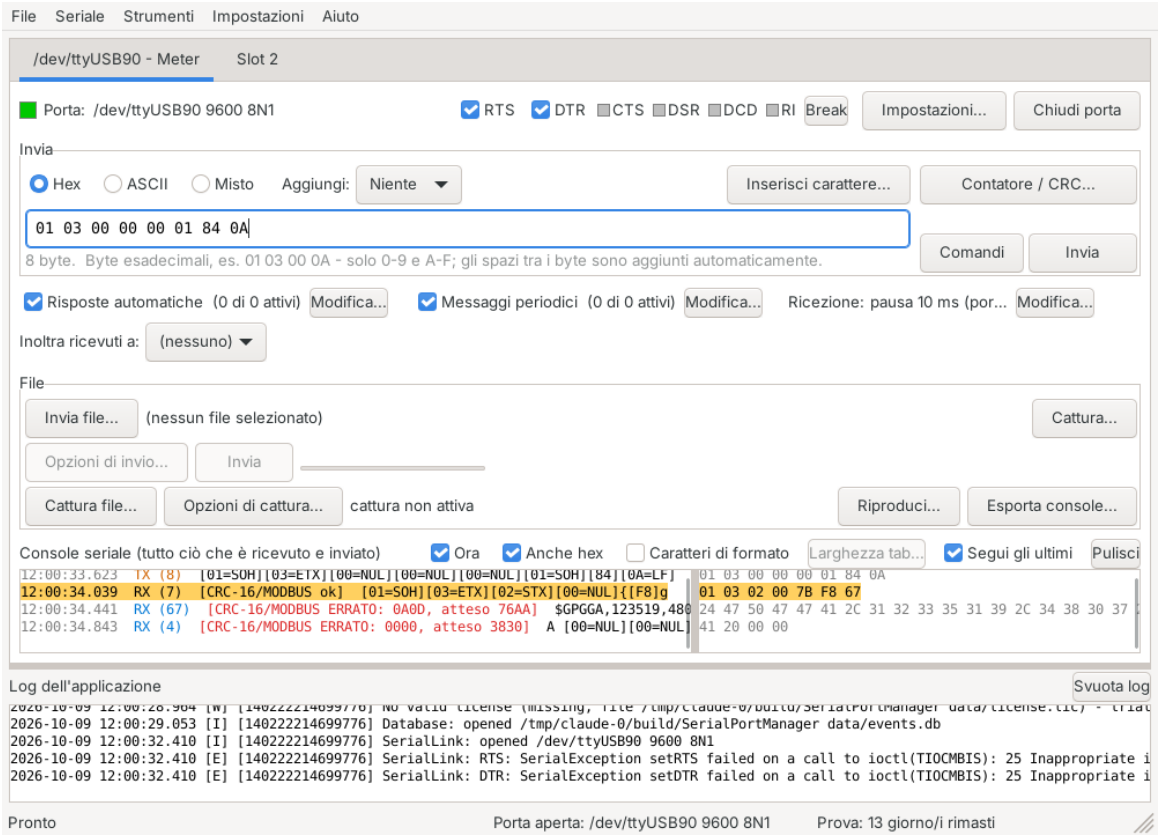
Per iniziare a lavorare:

1. Scegliere **Seriale > Porte seriali...** per decidere quante porte controllare e configurarle, oppure fare clic su **Impostazioni...** nella scheda per configurare solo la porta di quella scheda.
2. Fare clic su **Apri porta**.
3. Scrivere i dati nella casella **Invia** e premere Invio o fare clic su **Invia**.

La prova conta i giorni di calendario dal primo avvio su quel PC. Dopo la prova il programma continua a funzionare come monitor (le porte si aprono, tutto ciò che viene ricevuto è mostrato e registrato); invio, risposte automatiche, messaggi periodici, trasferimento file e inoltro richiedono una licenza (capitolo 19).

4. La finestra principale

Copyright © 2026 Fation Coga. Tutti i diritti riservati.



La finestra principale con una scheda porta, la console e il log dell'applicazione

La barra del titolo mostra il nome del programma, la versione e la data della build. Dall'alto in basso la finestra contiene:

- la barra dei menu: **File**, **Seriale**, **Strumenti**, **Impostazioni**, **Aiuto**;
- le schede delle porte, una per slot: il titolo di ogni scheda è la porta seguita dal nome assegnato, per esempio **COM3 - Contatore** (solo il nome, o **Slot N**, finché non è impostata alcuna porta);
- sotto le schede, separato da una barra trascinabile, il **Log dell'applicazione** con il pulsante **Svuota log**;
- la barra di stato.

4.1 Schede (slot)

Uno slot è una scheda con la propria porta, il proprio thread di lettura, la propria console, le proprie risposte automatiche e i propri messaggi periodici, la propria scelta di inoltro e il proprio trasferimento file. Gli slot lavorano contemporaneamente e in modo indipendente. Il numero di slot e i loro nomi si impostano con **Seriale > Porte seriali...** (capitolo 5). I comandi **Impostazioni porta...**, **Risposte automatiche...**, **Messaggi periodici...** e **Pulisci console** del menu **Seriale**, e **Strumenti > Analizzatore di byte...**, agiscono sulla scheda selezionata.

Ogni scheda contiene, dall'alto in basso:

1. **La riga della porta**: un LED di stato (verde = porta aperta, spento = chiusa, arancione = in attesa che una porta persa ritorni), **Porta:** seguito dalla porta e dalle sue impostazioni (per esempio *COM3 9600 8N1*, o **non impostata**; passandoci sopra si vede che dispositivo è, per esempio *USB-SERIAL CH340*), le linee di controllo **RTS**, **DTR**, **CTS**, **DSR**, **DCD**, **RI** e **Break** (6.2), il pulsante **Impostazioni...** (impostazioni della porta, capitolo 6) e il pulsante **Apri porta** / **Chiudi porta**.
2. **La casella Invia** (capitolo 7), con **Comandi** accanto a **Invia** (7.4).

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

3. **La riga dell'automazione**: gli interruttori **Risposte automatiche** e **Messaggi periodici**, ognuno con un riepilogo come *(2 di 3 abilitati)* e un pulsante **Modifica...**; **Ricezione:** con il modo in cui i byte ricevuti diventano messaggi e il suo **Modifica...** (8.4); **Inoltra ricevuti a:** (capitolo 10). La riga continua su una seconda riga quando la finestra è stretta.
4. **La casella File** (capitolo 11): **Invia file...**, il nome del file, **Opzioni di invio...**, **Invia** (o **Ferma**), una barra di avanzamento e il testo di avanzamento; **Cattura file...** (o **Ferma cattura**), **Opzioni di cattura...** e lo stato della cattura; a destra **Cattura...**, **Riproduci...** ed **Esporta console...** (8.8).
5. **La console**: **Console seriale (tutto ciò che è ricevuto e inviato)** in due riquadri (testo ed esadecimale) con le opzioni **Ora**, **Anche hex**, **Caratteri di formato**, **Larghezza tab...**, **Segui gli ultimi** e il pulsante **Pulisci** (capitolo 8).

4.2 Menu

Voce di menu	Tasti rapidi	Cosa fa
File > Esporta profilo...		salva le impostazioni delle porte in un file (capitolo 18)
File > Importa profilo...		le carica da un file (capitolo 18)
File > Esci	Alt-X	chiude il programma e le porte
Seriale > Porte seriali...		numero di porte, nomi, impostazioni
Seriale > Impostazioni porta...		porta della scheda attuale
Seriale > Risposte automatiche...		risposte automatiche della scheda
Seriale > Messaggi periodici...		messaggi periodici della scheda
Seriale > Pulisci console		svuota la console della scheda
Strumenti > Analizzatore di byte...	Ctrl-B	analizza i byte da inviare
Strumenti > Protocolli (informazioni e simulazione)...	Ctrl-P	la finestra Protocolli
Strumenti > Master Modbus...	Ctrl-M	la finestra del master Modbus (capitolo 15)
Strumenti > Plotter seriale...	Ctrl-L	il plotter seriale (capitolo 16)
Strumenti > Sequenze di test...	Ctrl-T	le sequenze di test (capitolo 17)
Impostazioni > Tema...		chiaro o scuro
Impostazioni > Lingua...		italiano o inglese
Impostazioni > Livello log...		quanto registra il log
Impostazioni > Database eventi...		limiti, esportazione e pulizia di events.db (20.5)
Impostazioni > Apri cartella dati		la cartella dati in Esplora risorse
Aiuto > Manuale utente	F1	questo manuale (PDF)
Aiuto > Guida rapida		la guida breve (PDF)
Aiuto > Licenza...	Ctrl-K	la finestra Licenza
Aiuto > Informazioni versione...		versione, build, cronologia
Aiuto > Informazioni su...		nome, copyright, librerie

4.3 Barra di stato

La barra di stato ha tre campi:

- a sinistra: messaggi generali, per esempio **Pronto** o **UID copiato negli appunti.**;
- al centro: la porta della scheda selezionata, **Porta aperta: COM3 9600 8N1** o **Porta chiusa**;
- a destra: lo stato della licenza, **Licenza: NOME**, **Licenza: NOME - scade tra N giorno/i** (negli ultimi 14 giorni), **Prova: N giorno/i rimasti**, oppure **Senza licenza: invio, file e inoltro disabilitati (Aiuto > Licenza)**.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

4.4 Log dell'applicazione

La parte inferiore della finestra mostra il log dell'applicazione: informazioni di avvio, cartella dati, lingua caricata, stato della licenza, problemi delle porte (per esempio quando un adattatore USB viene scollegato: la porta non si può leggere e il programma riprova ogni secondo) e altri eventi, nella forma *data ora [livello] [thread] messaggio*. Le righe del log sono scritte in inglese. La vista conserva le ultime 5000 righe; la cronologia completa è in log.txt nella cartella dati. **Svuota log** svuota solo la vista (non il file). Quanto viene registrato si imposta con **Impostazioni > Livello log...**.

5. Seriale > Porte seriali: quante porte, nomi e impostazioni

Seriale > Porte seriali... apre la finestra **Porte seriali**:

- In alto, le porte trovate in questo momento sul PC: **Porte seriali su questo PC: N (COM1, COM3, ...).** oppure **Nessuna porta seriale trovata su questo PC al momento.**
- **Quante porte seriali controllare:** il numero di slot (schede), da 1 al numero indicato in **(al massimo N)**. Il massimo è il numero di porte presenti sul PC (ma mai meno degli slot già esistenti), e mai più di 32.
- Il suggerimento: **Ogni slot è una scheda con la propria porta, thread e console. Una porta può appartenere a un solo slot.**
- Una riga per slot: **Slot N**, **Nome:** (testo libero; vuoto = la scheda prende il nome della porta o **Slot N**), le impostazioni attuali della porta (oppure **non configurato**) e **Configura...**, che apre la finestra delle impostazioni della porta del capitolo 6 per quello slot.

Fare clic su **OK** per applicare. Le schede vengono aggiunte o tolte in fondo; riducendo il numero, le ultime schede vengono chiuse (con le loro porte). Riducendo e poi aumentando di nuovo il numero nella stessa finestra si recuperano gli slot tolti, con le loro impostazioni. Le porte che passano da uno slot all'altro vengono liberate per prime, così una porta può passare da una scheda a un'altra in un solo passaggio.

5.1 Una porta appartiene a una sola scheda

Una porta COM può essere aperta da un solo programma e, dentro SerialPortManager, da un solo slot. Nell'elenco delle porte della finestra delle impostazioni, una porta già assegnata a un altro slot appare come **COM3 (già usata da Slot 2 - Contatore)**. Se la si sceglie comunque, il programma chiede **Porta già in uso**: *"COM3 è già usata da Slot 2 - Contatore. Scollegarla da quello slot e usarla qui, oppure annullare questa scelta?"*

- **Scollega e usa qui**: l'altro slot viene chiuso e perde la porta (la sua console mostra **Porta scollegata** in rosso); questo slot la prende.
- **Annulla scelta**: non cambia nulla; la porta resta all'altro slot.

6. Impostazioni della porta

La finestra **Porta seriale** si apre da **Impostazioni...** nella scheda, da **Seriale > Impostazioni porta...** (scheda attuale) oppure da **Configura...** in **Seriale > Porte seriali...**.

Campo	Valori	Significato
Porta	elenco delle porte trovate, oppure scritta a mano	la porta COM; **Aggiorna** rilegge l'elenco (per un adattatore USB appena collegato)
Velocità (baud)	da 1200 a 921600 dall'elenco, o qualsiasi valore scritto	bit al secondo; le due parti devono usare la stessa

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Campo	Valori	Significato
Bit di dati	5, 6, 7, 8	bit di dati per carattere (di solito 8)
Parità	None, Dispari, Pari, Mark, Space	il bit di parità opzionale
Bit di stop	1, 1.5, 2	durata del bit (o dei bit) di stop
Controllo di flusso	None, Software (XON/XOFF), Hardware (RTS/CTS)	come il ricevitore ferma il mittente
Timeout tra byte (ms)	0 - 60000	silenzio che chiude un messaggio ricevuto (vedi sotto)
Timeout lettura (ms)	0 - 60000	attesa massima di una lettura della porta
Timeout scrittura (ms)	0 - 60000	attesa massima per completare una scrittura
Riapri la porta quando ritorna (adattatore USB scollegato)	sì / no (sì)	vedere 6.3

Le impostazioni predefinite sono 9600 baud, 8 bit di dati, nessuna parità, 1 bit di stop (8N1), nessun controllo di flusso, timeout tra byte 10 ms, timeout di lettura e scrittura 100 ms. Un timeout di lettura o scrittura pari a 0 viene portato a 50 ms.

Come i byte ricevuti diventano messaggi. Una linea seriale trasporta byte, non messaggi. SerialPortManager raggruppa i byte ricevuti in una riga della console (un messaggio RX) finché la linea resta in silenzio per il **Timeout tra byte** (almeno 5 ms), oppure finché sono stati raccolti 4096 byte. Se i messaggi del dispositivo appaiono divisi su più righe RX, aumentare il timeout tra byte; se più messaggi appaiono uniti in una riga, diminuirlo (deve essere più breve della pausa tra due messaggi del dispositivo). Altri modi di dividere i byte in messaggi - un delimitatore come CR LF, una lunghezza fissa, una lunghezza indicata nel messaggio, la temporizzazione Modbus RTU - sono nelle opzioni di **Ricezione** della scheda (8.4).

Timeout di scrittura. Un messaggio lungo a bassa velocità richiede più tempo del timeout di scrittura: il programma lo scrive a fette, ognuna delle quali richiede al massimo metà del timeout, così i messaggi lunghi e i blocchi dei file non vengono troncati. Una scrittura che comunque non si completa (per esempio con controllo di flusso hardware e l'altra parte non pronta) viene segnalata nella console come **Scrittura non riuscita (...)**.

Facendo clic su **OK** con la porta aperta, la porta viene chiusa e riaperta con le nuove impostazioni. Se la porta è ora usata da un altro slot, viene posta la domanda descritta in 5.1.

6.1 Apertura e chiusura

Apri porta apre la porta con le sue impostazioni. Il LED diventa verde, il pulsante diventa **Chiudi porta**, il campo centrale della barra di stato mostra **Porta aperta: ...**, la casella di invio diventa utilizzabile e le risposte automatiche e i messaggi periodici attivi partono. Se la porta non si può aprire (usata da un altro programma, scollegata, nome errato) un errore indica **Impossibile aprire COM3:** seguito dal motivo fornito da Windows. **Apri porta** è disattivato finché non è impostata alcuna porta.

Chiudi porta ferma il thread di lettura, i messaggi periodici e un eventuale file in invio (che si potrà recuperare in seguito, capitolo 11.6) e chiude la porta. I messaggi ancora in attesa di invio vengono scartati.

Se un adattatore USB-seriale viene scollegato mentre la porta è aperta, la porta viene chiusa e riaperta quando ritorna (6.3).

6.2 Linee di controllo e Break

La riga della porta ha le linee di controllo modem della porta:

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- **RTS** e **DTR** - le due uscite, spuntata = attiva. Il loro stato è ricordato per scheda e impostato ogni volta che la porta si apre; si possono cambiare anche a porta chiusa. **RTS** è disabilitata con il controllo di flusso **Hardware (RTS/CTS)** (allora la comanda il driver). Molte schede si resettano quando DTR cambia (Arduino, ESP32); RTS spesso commuta un transceiver RS-485 in trasmissione.
- **CTS**, **DSR**, **DCD**, **RI** - i quattro ingressi, mostrati da LED (verde = attivo), letti dieci volte al secondo a porta aperta.
- **Break** - tiene la linea TX nello stato di break (0) per 250 ms: un reset o un marcatore di trama per alcuni dispositivi (LIN, DMX512, bootloader).

Ogni cambiamento è annotato nella console (etichetta **Linee**), in rosso se la porta o il suo driver lo ha rifiutato (le porte virtuali spesso non hanno linee modem).

6.3 Porte perse e descrizioni

Quando la porta smette di funzionare mentre è aperta - tipicamente un adattatore USB-seriale scollegato - la console dice **COM3 persa (scollegata?) - chiusa; verrà riaperta quando ritorna**, il LED diventa arancione e il pulsante diventa **Smetti di attendere**. Ogni secondo il programma cerca la porta; appena Windows la elenca di nuovo viene aperta con le stesse impostazioni (**COM3 è tornata: riaperta**), automazione e linee di controllo comprese. **Smetti di attendere** rinuncia. Il comportamento è impostato da **Riapri la porta quando ritorna (adattatore USB scollegato)** nelle impostazioni della porta (attivo per impostazione predefinita); disattivato, il programma ritenta solo la lettura ogni secondo.

L'elenco delle porte nelle impostazioni mostra che cos'è ogni dispositivo, come lo chiama Windows, per esempio **COM3 (USB-SERIAL CH340)**, o il suo identificativo hardware (*USB VID:PID=1A86:7523*); il testo della porta di una scheda lo mostra passandoci sopra.

7. La casella Invia

La casella **Invia** di ogni scheda contiene i byte da inviare a mano. Sopra il testo ci sono i pulsanti di notazione **Hex**, **ASCII** e **Misto**, l'elenco **Aggiungi:** e il pulsante **Inserisci carattere...**; sotto, una riga di suggerimento mostra il numero di byte e come scriverli, oppure - in rosso - dove il testo è sbagliato e perché. A destra, **Contatore / CRC...** imposta ciò che Invia inserisce nel messaggio (sezione 7.3), con sotto un riepilogo come *+ contatore 1 B, prossimo 5 (ultimo), CRC-16/MODBUS (ultimo)*, e il pulsante **Invia**.

Premere **Invio** nel testo o fare clic su **Invia** per inviare. La casella e il pulsante **Invia** sono attivi solo con la porta aperta (e **Invia** è disattivato durante l'invio di un file). Se il testo non è valido compare un messaggio come **Carattere 7: sono ammesse solo cifre esadecimali (0-9, A-F).** e il carattere sbagliato viene selezionato. L'invio richiede la funzione di licenza *Invio*.

Aggiungi: aggiunge **Niente**, **CR**, **LF** o **CR e LF** alla fine di ogni messaggio inviato. Il testo nella casella resta come scritto; la console mostra i byte come sono stati inviati, fine riga compreso.

Il testo, la notazione, la scelta di **Aggiungi:** e le opzioni contatore / CRC di ogni scheda vengono ricordati alla chiusura del programma (il contatore riprende dal suo valore successivo).

7.1 Notazioni

Notazione	Come scrivere	Esempio
Hex	coppie di cifre esadecimali; gli spazi tra i byte sono aggiunti automaticamente	`01 03 00 0A`
ASCII	solo caratteri ASCII stampabili (da spazio a ~)	`CIAO`
Misto	testo, e `//0x` seguito da due cifre esadecimali per qualsiasi byte	`OK//0x0D//0x0A`

Dettagli:

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- **Hex**: si possono scrivere solo 0-9 e A-F (le minuscole diventano maiuscole); il testo viene rispaziato mentre si scrive. I byte incollati in stile C come `0x01, 0x03` vengono ripuliti: i prefissi `0x` e gli altri caratteri vengono tolti. Un byte con una sola cifra è un errore: **questo byte ha una sola cifra esadecimale - ne servono due.**
- **ASCII**: i byte di controllo (CR, LF, 00...) non si possono scrivere. Per questi usare Misto.
- **Misto**: un `//0x` per ogni byte, per esempio CR LF si scrive `//0x0D//0x0A`. `//0x` deve essere seguito da due cifre esadecimali: **//0x deve essere seguito da due cifre esadecimali, es. //0x0D.** Un testo letterale "//0x" nei dati viene scritto con la prima barra come `//0x2F` (cioè `//0x2F/0x`): il programma lo fa da sé nelle conversioni.

Facendo clic su un'altra notazione il testo attuale viene convertito: i byte restano gli stessi. Se si passa ad ASCII e i dati contengono byte che ASCII non può mostrare, il programma chiede se passare invece a Misto (**No** mantiene la notazione attuale).

7.2 Tabella Inserisci carattere

Inserisci carattere... apre la tabella **Inserisci carattere** con tutti i 256 valori di byte: **Dec**, **Hex**, **Car.**, **Nome** (per esempio NUL, STX, CR, LF, ESC, DEL) e **Descrizione** (per esempio *Ritorno carrello*, *Lettera maiuscola A*, *Non ASCII (mostrato come carattere Latin-1)* per 80-FF).

- Doppio clic su un carattere per aggiungerlo alla posizione del cursore e chiudere la tabella.
- Clic destro e **Aggiungi X e tieni aperta la tabella**, oppure selezione e **Aggiungi, tieni aperto**, per aggiungere più caratteri; la riga **Aggiunti: ...** mostra cosa è stato aggiunto.
- **Chiudi** chiude la tabella.

Il carattere viene scritto nella notazione in uso: in Hex come due cifre, in Misto come carattere o come `//0xHH`; in ASCII, aggiungere un byte di controllo fa passare prima il testo a Misto.

7.3 Contatore / CRC...

Contatore / CRC... apre la finestra **Contatore e checksum**. Ciò che imposta viene INSERITO da Invia nel messaggio scritto; il testo nella casella non cambia mai.

Contatore - **Inserisci un contatore incrementale**:

- **Byte**: da 1 a 8 (predefinito 1).
- **Valore successivo**: il valore del prossimo messaggio (decimale, `0x` esadecimale o `0b` binario).
- **Aggiunto a ogni invio**: aggiunto dopo ogni messaggio inviato; negativo conta all'indietro. Oltre il valore massimo dei suoi byte riparte da 0 (255 + 1 = 0 con un byte).
- **Ordine dei byte** (con più di un byte): **Little-endian (prima il byte meno significativo)** - il predefinito - o **Big-endian (prima il byte più significativo)**.
- **Posizione** e **Byte di distanza**: **Contati dalla fine** - 0 = ultimo, 1 = prima dell'ultimo byte, 2 = prima degli ultimi due... - oppure **Contati dall'inizio** - 0 = primo, 1 = dopo il primo byte...

Checksum / CRC - **Inserisci un checksum / CRC del messaggio**:

- **Algoritmo**: tutti i checksum e i CRC dell'Analizzatore di byte (somme, XOR, LRC, Fletcher, Adler, da CRC-8 a CRC-64). Sceglierne uno imposta l'ordine dei byte con cui di solito si invia (per esempio CRC-16/MODBUS little-endian).
- **Ordine dei byte**, **Posizione**, **Byte di distanza**: come per il contatore.
- **Includi il contatore nel calcolo**: se i byte del contatore sono coperti. Quando contatore e checksum sono entrambi attivi, il programma chiede *Includere il contatore nel calcolo del checksum / CRC?*

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Il messaggio si costruisce in quest'ordine: si inserisce il contatore, si calcola il checksum (sui byte scritti, con o senza il contatore) e lo si inserisce, poi si aggiunge il fine riga di **Aggiungi:** - il fine riga non fa mai parte del checksum.

Il riquadro **Prossimo messaggio** mostra in anteprima il prossimo messaggio con il testo e il fine riga attuali: il contatore tra `[ ]`, il checksum tra `{ }`, per esempio `01 03 00 00 00 01 [07] {4B A1} 0D 0A`, e il valore del contatore dopo di esso.

Esempio - una richiesta Modbus RTU con il suo CRC: testo `01 03 00 00 00 01`, checksum CRC-16/MODBUS, ultimo, little-endian: Invia scrive `01 03 00 00 00 01 84 0A`.

Lo stesso campo dati (notazioni, suggerimento, tabella dei caratteri) è usato ovunque il programma chieda dei byte: schemi e risposte automatiche, messaggi periodici, comandi dei file, Analizzatore di byte e finestra Protocolli.

7.4 Comandi e cronologia

Su e **Giù** nel testo da inviare richiamano i messaggi inviati da questa scheda, dal più recente (gli ultimi 50, conservati tra le sessioni), con la loro notazione e **Aggiungi:**; **Giù** oltre il più recente torna a ciò che si stava scrivendo. Inviare di nuovo un messaggio lo porta in cima.

Comandi, accanto a **Invia**, apre un menu:

- i **comandi salvati**: un clic ne carica uno nella casella di invio e lo invia (le opzioni contatore / CRC della scheda si applicano come per **Invia**);
- **Salva il testo come comando...** - chiede un nome (lo stesso nome sostituisce un comando);
- **Elimina comando** - un sottomenu dei comandi salvati;
- **Inviati prima** - la cronologia: un clic carica un messaggio senza inviarlo;
- **Cancella la cronologia**.

I comandi salvati sono comuni a tutte le schede.

8. La console

La console di ogni scheda, **Console seriale (tutto ciò che è ricevuto e inviato)**, ha due riquadri: i messaggi come testo a sinistra e, con **Anche hex**, gli stessi byte in esadecimale a destra - riga per riga, ogni riga esadecimale accanto alla riga di testo a cui appartiene. I due riquadri scorrono insieme (scorrere l'uno sposta l'altro) e il divisore tra loro si può trascinare (la sua posizione viene ricordata). La console conserva le ultime 5000 righe, la più vecchia in alto.

8.1 Righe

Una riga di messaggio ha la forma:

```
14:03:21.512  RX (6)  [06=ACK]OK 1[0D=CR]        06 4F 4B 20 31 0D
```

- l'ora con i millisecondi (opzione **Ora**);
- l'etichetta e, tra parentesi, il numero di byte;
- i byte come testo: i caratteri stampabili così come sono, i byte di controllo come `[HH=NOME]` (per esempio `[0D=CR]`), gli altri byte (oltre 7E) come `[HH]`;
- nel riquadro di destra, i byte di questa riga in esadecimale (opzione **Anche hex**).

Con **Caratteri di formato** attivo, LF, CR, CR e LF, tabulazione verticale (0B) e avanzamento pagina (0C) iniziano una nuova riga: un messaggio di più righe di testo viene mostrato su più righe della console, le successive sotto l'inizio della prima (senza ripetere ora ed etichetta), ognuna con i propri byte nel riquadro

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

esadecimale (fine riga compreso). Una tabulazione (09) diventa spazi. Ogni altro carattere di controllo resta visibile come `[HH=NOME]`, ogni byte oltre 7E come `[HH]`.

Etichetta	Colore mostrato	Significato
RX	blu	ricevuto
TX	arancione	inviato dalla casella di invio
TX auto "nome"	viola	inviato dalla risposta automatica *nome*
TX periodico "nome"	verde	inviato dal messaggio periodico *nome*
TX inoltro "scheda"	verde acqua	ricevuto dalla scheda *scheda* e inoltrato da questa porta
TX prefisso file	ambra	prefisso iniziale di un file (capitolo 11)
TX fine file	ambra	marcatore di fine di un file
TX recupero file	ambra	comando di recupero di un file
TX risposta file	ambra	risposta inviata durante la cattura di un file
File selezionato	ambra	è stato scelto un file
Invio file	ambra	inizia un trasferimento; ogni blocco è una riga **TX** come *blocco 3/10 (64 byte)*
Recupero file	ambra	continua un trasferimento interrotto
File	ambra	esiti del trasferimento e note della cattura
TX replay	verde	inviato dalla Riproduzione (8.8)
TX Modbus / TX Sequence	verde acqua	inviato dal master Modbus (capitolo 15) o da una sequenza di test (capitolo 17)
Linee	verde acqua	una linea di controllo è cambiata o è stato inviato un break (6.2)
Porta	rosso / verde acqua	la porta è stata persa / è tornata (6.3)
Cattura, Riproduzione, Esportazione	verde acqua / verde	note di cattura, riproduzione ed esportazione (8.8)
qualsiasi etichetta	rosso	un errore: **Scrittura non riuscita (...)**, **Troppi messaggi in attesa - questo non è stato inviato.**, una risposta automatica o un messaggio periodico che non si può costruire, **Porta scollegata**, un inoltro non eseguito, un trasferimento file fallito

La nota rossa **Risposte automatiche e messaggi periodici non avviati: richiedono una licenza (Aiuto > Licenza).** compare quando gli interruttori sono attivi ma la licenza non comprende quelle funzioni.

8.2 Opzioni della console

- **Ora**: mostra l'ora all'inizio di ogni riga.
- **Anche hex**: mostra il riquadro esadecimale (è sempre riempito: attivandolo si vedono anche le righe precedenti).
- **Caratteri di formato**: i fine riga iniziano una nuova riga e le tabulazioni diventano spazi (vedi sopra). Attivandolo il programma chiede quanti spazi occupa una tabulazione; **Larghezza tab...** la cambia (da 1 a 16).
- **Segui gli ultimi**: tiene in vista l'ultima riga - in entrambi i riquadri, poiché scorrono insieme. Disattivarlo per leggere le righe precedenti mentre continuano ad arrivare dati.
- **Pulisci**: svuota entrambi i riquadri (anche **Seriale > Pulisci console** e il menu del clic destro).

Le opzioni sono comuni a tutte le schede e vengono ricordate; **Ora** e **Caratteri di formato** valgono per le righe scritte dopo la modifica.

8.3 Suggerimento

Passando con il mouse sulla console compare il suggerimento **Clic destro su un messaggio - o seleziona alcuni dei suoi byte, nel testo o nella parte esadecimale - per checksum, numeri,**

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

simulazione del segnale e decodifica del protocollo. Il menu del clic destro è descritto nel capitolo 12.

8.4 Opzioni di ricezione: suddivisione e verifica del checksum

Ricezione: nella riga dell'automazione mostra come i byte ricevuti diventano messaggi (per esempio *pausa 10 ms (porta)*, *finisce con 0D 0A, CRC-16/MODBUS*); **Modifica...** apre le **Opzioni di ricezione**.

I messaggi finiscono:

Scelta	Un messaggio finisce	Impostazioni
dopo una pausa (intervallo)	quando non arriva alcun byte per l'intervallo	**Intervallo (ms):** 0 = il timeout tra byte della porta
con un delimitatore (es. CR LF)	con i byte delimitatori	**Delimitatore (hex):** per esempio `0D 0A`, `0A`, `03`; **Mostra il delimitatore**
dopo un numero fisso di byte	ogni N byte	**Byte per messaggio:** 1 - 4096
dopo la lunghezza indicata nel messaggio	a scostamento + campo + valore + correzione byte	**Lunghezza al byte:**, **Byte della lunghezza:** 1 - 4, **Ordine dei byte**, **Correzione:**
Modbus RTU (3,5 caratteri di silenzio)	dopo 3,5 tempi di carattere di silenzio alla velocità della porta (1,75 ms oltre 19200 bit/s)	-

Tranne che con l'intervallo, un messaggio che resta incompleto viene comunque mostrato dopo **Timeout (ms):**, così nulla va perso quando il dispositivo non rispetta la suddivisione. Ogni messaggio è mostrato al più tardi a 4096 byte. Esempio - trame *STX, lunghezza (2 byte, little-endian), dati, un checksum di 1 byte*: **Lunghezza al byte** 1, **Byte della lunghezza** 2, little-endian, **Correzione** +1. Con gli adattatori USB la temporizzazione Modbus RTU dipende dalla latenza dell'adattatore: se le trame arrivano spezzate o unite, usare un intervallo di qualche ms.

Verifica dei messaggi ricevuti - **Verifica il checksum / CRC di ogni messaggio ricevuto**: l'**Algoritmo** (ognuno dell'Analizzatore di byte), il suo **Ordine dei byte**, la sua **Posizione** (**dalla fine**: **Scostamento** = i byte dopo di esso, 0 = ultimo, 2 = segue un CR LF; **dall'inizio**: il byte dove comincia) e **Non coperti all'inizio:** (es. 1 per un STX che non ne fa parte). Il checksum copre i byte prima di esso. Ogni riga RX mostra allora il risultato subito dopo l'etichetta: **[CRC-16/MODBUS ok]** in verde, o in rosso **[CRC-16/MODBUS ERRATO: 0A84, atteso 0B94]** o **[troppo corto per CRC-16/MODBUS]**.

8.5 Cercare, filtrare ed evidenziare

Ctrl-F nella console (o **Trova...** nel suo menu del clic destro) apre la barra di ricerca sotto le opzioni della console: scrivere il testo; **Invio** / **>** trova la corrispondenza successiva, **Maiusc-Invio** / **<** la precedente, **F3** / **Maiusc-F3** nella console fanno lo stesso; la barra dice *3 di 12* o *non trovato*. **Maiuscole/minuscole**; **Nell'hex** cerca nel riquadro hex (per esempio `0D 0A`); **Solo le righe trovate** mostra solo le righe che contengono il testo. **Esc** o **x** chiude la barra (e termina quel filtro).

Il menu del clic destro **Mostra** accende e spegne **Ricevuti (RX)**, **Inviati (TX, risposte automatiche, periodici, inoltrati, riproduzione)** e **Note (porta, file, cattura...)** (per scheda, ricordato). Mentre qualcosa è nascosto il titolo della console dice **Console seriale - filtrata: 12 di 40 righe**. Le righe nascoste sono conservate: riaccendendole ricompaiono al loro posto.

Regole di evidenziazione... (menu del clic destro): i messaggi che contengono un testo (senza distinguere maiuscole, es. *ERROR*) o una sequenza di byte (**Byte hex**, es. `01 83`) sono mostrati su un colore, in entrambi i riquadri e in tutte le schede. **Aggiungi**, **Sostituisci**, **Cancella**, **Sposta su**; la spunta accende e spegne una regola; vince la prima regola spuntata che corrisponde. Il testo è nero o bianco, quello che si legge meglio sul colore.

8.6 Tempo tra i messaggi

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Tempo dal messaggio precedente (delta) nel menu del clic destro aggiunge, dopo l'orario, i secondi dal messaggio precedente della scheda, per esempio `14:03:21.512 +0.302 RX (8) ...` - per misurare tempi di risposta e periodi.

8.7 Statistiche

Statistiche... (menu del clic destro) mostra, aggiornati due volte al secondo: **Conteggio dal**, **Porta aperta da**, **Ricevuti** e **Inviati** (byte e messaggi), **Ricezione ora** / **Invio ora** (byte al secondo), **Errori di scrittura** e **Checksum dei ricevuti** (ok, errati, troppo corti - con la verifica di 8.4). **Azzera** ricomincia il conteggio. I dati di un file in cattura non sono contati come ricevuti; gli errori di trama, parità e overrun della linea non sono riportati dalla libreria seriale.

8.8 Cattura, riproduzione ed esportazione

Cattura... (casella File) scrive il traffico della scheda in un file mentre avviene, fino a **Ferma cattura**:

- **Cattura temporizzata (`*.spmcap`)**: ogni messaggio ricevuto e inviato, con il suo orario - per la Riproduzione;
- **Byte ricevuti grezzi (`*.bin`)**: i byte ricevuti uno dopo l'altro - per altri strumenti.

La dimensione catturata è mostrata accanto al pulsante; la console annota l'inizio e la fine.

Riproduci... invia di nuovo i messaggi di una cattura temporizzata, con le loro pause: **ciò che è stato inviato (ripete una sessione)** o **ciò che è stato ricevuto (simula l'altro dispositivo)**, a **1x**, **2x**, **5x**, **10x** o **senza pause** (le pause oltre 10 s vengono ridotte a 10 s). I messaggi compaiono come **TX replay**; **Ferma riproduzione** la termina. La riproduzione richiede la funzione di licenza *Invio*.

Esporta console... (anche nel menu del clic destro) salva la console come appare (vale il filtro): **Testo (`*.txt`), le righe come sono, o** CSV (`*.csv`)*, una riga per messaggio con *orario, etichetta, byte, hex, testo, nota* (UTF-8 per i fogli di calcolo).

9. Risposte automatiche e messaggi periodici

9.1 Interruttori ed elenchi

Ogni scheda ha due interruttori nella riga dell'automazione:

- **Risposte automatiche**: se spuntato, le risposte automatiche attive di questa scheda funzionano mentre la porta è aperta.
- **Messaggi periodici**: se spuntato, i messaggi periodici attivi di questa scheda vengono inviati mentre la porta è aperta.

Accanto a ogni interruttore, **(N di M attivi)** indica quanti elementi sono attivi e **Modifica...** apre l'elenco. Spuntando un interruttore senza la funzione di licenza necessaria, il programma chiede se aprire la finestra Licenza.

Seriale > Risposte automatiche..., **Seriale > Messaggi periodici...** e i pulsanti **Modifica...** aprono la finestra **Risposte automatiche e messaggi periodici**, con due schede:

- **Risposte automatiche**: *Con la porta aperta, ogni regola spuntata risponde appena riceve i suoi byte. Doppio clic per modificare.* Colonne **Nome**, **Quando si riceve**, **Risposta**, **Attesa (ms)**.
- **Messaggi periodici**: *Con la porta aperta, ogni messaggio spuntato è inviato al suo intervallo. Doppio clic per modificare.* Colonne **Nome**, **Messaggio**, **Ogni (ms)**.

La casella di spunta all'inizio di ogni riga attiva o disattiva l'elemento. I pulsanti a destra: **Aggiungi...**, **Modifica...** (o doppio clic), **Duplica** (una copia sotto l'elemento selezionato) e **Rimuovi** (chiede **Rimuovere "nome"?**). Le modifiche hanno effetto facendo clic su **OK** (e vengono scartate con **Annulla**);

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

vengono salvate con la scheda.

9.2 Risposta automatica

La finestra **Risposta automatica** contiene:

- **Nome:** (predefinito *Risposta automatica N*) e **Attivo**;
- **Quando si ricevono questi byte**: lo schema, in Hex, ASCII o Misto. Non può essere vuoto (**Scrivi i byte da riconoscere.**);
- **Rispondi con**: l'editor del messaggio (9.4);
- **Attesa prima di rispondere (ms):** da 0 a 600000 ms; 0 risponde subito.

Funzionamento:

- Lo schema viene cercato nel flusso dei byte ricevuti, anche quando arriva diviso su più letture o in mezzo a un messaggio più lungo. Ogni occorrenza fa partire la risposta, e più regole possono rispondere agli stessi byte.
- Senza attesa viene mostrata prima la riga RX che contiene lo schema, poi la risposta; con un'attesa, la risposta viene inviata trascorsa l'attesa (nel frattempo si possono inviare e ricevere altri messaggi).
- La risposta appare come **TX auto "nome"** in viola.
- Le risposte automatiche si sospendono mentre questa scheda invia un file (le conferme del ricevitore non devono attivarle) e non vedono i byte di un file in cattura.
- I valori incrementali di una risposta ripartono dal valore iniziale quando la porta viene aperta o l'elenco viene modificato.
- Una risposta che non si può costruire (per esempio un testo non valido modificato a mano) viene segnalata in rosso: **La risposta automatica "nome" non può essere costruita - controlla il suo testo.**

9.3 Messaggio periodico

La finestra **Messaggio periodico** contiene **Nome:** (predefinito *Messaggio periodico N*), **Attivo**, **Invia ogni (ms):** (da 10 a 86400000 ms, cioè fino a 24 ore) e l'editor **Messaggio** (9.4).

Ogni messaggio attivo ha il proprio timer; il primo messaggio viene inviato un intervallo dopo l'apertura della porta (o dopo la modifica dell'elenco). Appare come **TX periodico "nome"** in verde. Durante l'invio di un file i messaggi periodici vengono saltati (finirebbero in mezzo al file). Un messaggio che non si può costruire viene fermato e segnalato una volta: **Il messaggio periodico "nome" non può essere costruito - controlla il suo testo. È stato fermato.**

Se più di 1000 messaggi attendono di essere scritti (per esempio periodi molto brevi a bassa velocità), i nuovi vengono rifiutati con **Troppi messaggi in attesa - questo non è stato inviato.**

9.4 L'editor del messaggio

Risposte, messaggi periodici e comandi e risposte del trasferimento file si modificano con lo stesso editor:

1. I byte fissi, in Hex, ASCII o Misto (con **Inserisci carattere...**).
2. **Valori incrementali (scritti sopra i byte del messaggio):** un elenco di contatori con **Aggiungi...**, **Modifica...** (o doppio clic) e **Rimuovi**. Ogni riga descrive il contatore, per esempio *Byte 2-3 (Numero binario, big-endian): da 0 a 65535, passo 1, 1 passi per messaggio inviato*.
3. **Checksum:** un riepilogo (per esempio *CRC-16/MODBUS, 2 byte a la fine, little-endian*, oppure *nessuno*) e il pulsante **Checksum...**.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

4. Il suggerimento: *un valore incrementale viene scritto SOPRA i byte fissi al suo indice - se il testo fisso ha un byte in quella posizione viene sostituito, quindi metti un segnaposto (un byte "indifferente") in quella posizione. Il checksum viene INSERITO al suo indice e sposta i byte successivi.*
5. **Anteprima dei primi messaggi (a un secondo di distanza):** i primi quattro messaggi come verranno inviati, per esempio `#1 (8 byte): 01 03 00 00 00 01 84 0A | ...`, così si vedono anche i contatori che avanzano al secondo.

Il messaggio viene costruito in quest'ordine: byte fissi, poi i valori incrementali scritti sopra, poi il checksum calcolato e inserito. Un messaggio vuoto viene rifiutato: **Scrivi il messaggio da inviare.**

9.5 Valori incrementali

La finestra **Valore incrementale**:

Campo	Significato
Scritto come:	**Numero binario** (1-8 byte), **Cifre decimali ASCII** (1-20 caratteri, con zeri iniziali) o **Cifre esadecimali ASCII** (1-16 caratteri, maiuscole)
Dimensione:	numero di byte (si vedono i bit: *byte (16 bit)*) o di **caratteri**
Inizia all'indice di byte:	il primo byte occupato (0 = primo byte del messaggio); la riga sotto indica quali byte occupa e se il messaggio va esteso con byte a zero
Ordine dei byte:	**Big-endian (prima il byte più significativo)** o **Little-endian (prima il byte meno significativo)**, per i valori binari di 2 o più byte
Valore iniziale:, Valore finale:	decimale, o esadecimale come 0x1F
Passo:	positivo, o negativo per contare all'indietro (da un valore iniziale maggiore del finale)
Avanza di:	un numero di **passi per messaggio inviato** o **al secondo**; sono ammesse frazioni: 0.5 per messaggio = un passo ogni 2 messaggi

Big-endian è il predefinito: Modbus e la maggior parte dei protocolli binari inviano i numeri così. I valori vanno dal valore iniziale con il passo indicato, mai oltre il valore finale, poi ripartono dal valore iniziale. Le righe informative mostrano i primi valori, la frequenza (per esempio *un passo ogni 2 messaggi inviati*) e **Valore massimo per questa dimensione: ...**. Gli errori vengono spiegati (dimensione fuori intervallo, valori che non stanno nella dimensione, passo 0, passo nella direzione sbagliata, numero di passi non maggiore di 0).

Per una risposta che il programma **attende** (trasferimento file, capitolo 11), un valore incrementale avanza di un passo per ogni risposta attesa, così si possono riconoscere risposte come *ACK 01*, *ACK 02*...

9.6 Checksum

La finestra **Checksum**:

- **Aggiungi un checksum al messaggio**: attiva il checksum.
- **Includi anche il messaggio a cui risponde: ...** (solo per le risposte, vedi sotto), con **Iniziarlo con il prefisso iniziale del file (poi il blocco / gruppo)** (solo conferme dei file e risposte ai gruppi) e **Escludere da esso gli indici:** (per esempio `0-2, -1`; -1 = l'ultimo byte).
- **Tipo:** uno dei 12 tipi sotto, mostrato come *nome - N byte*. Scegliendo un tipo viene scelto anche il suo ordine dei byte abituale.
- **Inserisci all'indice:** dove va il checksum (0 = prima del primo byte, N = dopo l'ultimo), oppure **alla fine (segue la lunghezza del messaggio)**.
- **Calcolato su: tutti i byte** oppure **tutti i byte tranne gli indici:** (per esempio `0, 5-7`; 0 = primo byte) - per esempio per escludere un carattere iniziale.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- **Byte da inviare:** quanti byte del checksum vengono inviati (**di N**), e se sono meno: **parte bassa (byte meno significativi)** o **parte alta (byte più significativi)**.
- **Ordine dei byte:** big-endian o little-endian.
- L'anteprima: **Checksum del messaggio attuale: 0x.... - inviato come .. .. all'indice N.** e il messaggio completo.

Tipo	Byte	Ordine abituale
SUM-8 (add)	1	-
SUM-16 (add)	2	big-endian
XOR-8 (BCC)	1	-
LRC-8 (two's complement of the sum)	1	-
CRC-8 (SMBus)	1	-
CRC-8/MAXIM (1-Wire)	1	-
CRC-16/MODBUS	2	little-endian
CRC-16/ARC (IBM)	2	little-endian
CRC-16/CCITT-FALSE	2	big-endian
CRC-16/XMODEM	2	big-endian
CRC-32 (IEEE, zip)	4	little-endian
CRC-32C (Castagnoli)	4	little-endian

I nomi dei tipi di checksum compaiono come nella tabella in tutte le lingue. Il checksum viene calcolato sul messaggio dopo la scrittura dei valori incrementali, senza gli indici esclusi, e viene ricalcolato per ogni messaggio inviato.

Includere il messaggio a cui si risponde. Alcuni dispositivi calcolano il checksum di una risposta anche sulla richiesta. Per le risposte, il checksum può includere anche *il messaggio a cui risponde*: i suoi byte vengono presi per primi (senza gli indici esclusi; gli indici negativi contano dalla fine e un indice che un messaggio più corto non ha viene ignorato), poi i byte della risposta (**Calcolato su** vale solo per la risposta). Che cos'è "il messaggio a cui risponde" dipende da dove si trova l'editor:

Editor	Il messaggio a cui risponde
risposta automatica	la riga RX ricevuta, fino alla fine dello schema
invio file: risposta al prefisso	il prefisso iniziale inviato
invio file: conferma	il blocco inviato (i suoi byte di dati)
invio file: risposta al recupero	il comando di recupero inviato
cattura: risposta al prefisso	il prefisso iniziale ricevuto
cattura: risposta ai gruppi	il gruppo di byte del file ricevuto
cattura: risposta al recupero	il comando di recupero ricevuto

L'anteprima indica allora **(anteprima senza il messaggio a cui risponde: si conosce solo quando arriva)**. I messaggi periodici non rispondono a nulla e quindi non hanno questa opzione.

10. Inoltro

Inoltra ricevuti a: a destra della riga dell'automazione sceglie un'altra scheda: tutto ciò che questa porta riceve viene inviato dalla porta di quella scheda. **(nessuno)** disattiva l'inoltro. L'elenco mostra le altre schede con il loro titolo ed è disattivato quando c'è una sola scheda.

- I dati inoltrati compaiono nella scheda di destinazione come **TX inoltro "scheda di origine"** in verde acqua; la scheda di origine li mostra come normale RX.
- Se la porta di destinazione è chiusa, la console di origine mostra in rosso **non inoltrato: la porta di SCHEDA è chiusa**.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- Se la scheda di destinazione sta inviando un file, o troppi messaggi sono in attesa, la console di destinazione mostra i dati in rosso con **Troppi messaggi in attesa - questo non è stato inviato.**
- L'inoltro va in una sola direzione. Per collegare due porte in entrambe le direzioni (un "ponte" tra due dispositivi, con SerialPortManager in ascolto nel mezzo), impostare ognuna delle due schede per inoltrare all'altra.
- L'inoltro richiede la funzione di licenza *Inoltro a un'altra porta*; se la licenza non la comprende più, l'inoltro torna a (nessuno).

11. Trasferimento file

Il riquadro **File** di ogni scheda invia un file attraverso la porta e cattura i file inviati nello stesso modo, per esempio per caricare firmware o dati in un dispositivo o per riceverne dei log. Il trasferimento file richiede la funzione di licenza *Trasferimento file*.

11.1 Inviare un file

1. **Invia file...** sceglie il file (fino a 256 MB). Nome e dimensione compaiono accanto al pulsante e la console mostra **File selezionato**.
2. **Opzioni di invio...** imposta come viene inviato (11.2).
3. Con la porta aperta, **Invia** avvia il trasferimento. Il pulsante diventa **Ferma**, la barra mostra l'avanzamento e il testo mostra *inviati / totale*. Prima dell'avvio il testo mostra il tempo minimo, **almeno X**; alla fine, **inviato in X**.
4. **Ferma** annulla il trasferimento.

Durante l'invio di un file il pulsante **Invia** della casella di invio è disattivato, i messaggi periodici vengono saltati, le risposte automatiche si sospendono e i dati inoltrati a questa scheda vengono rifiutati.

La console mostra **Invio file** (o **Recupero file**), **TX prefisso file**, una riga per blocco (**TX** *blocco 3/10 (64 byte)*, con *(ripetizione)* quando un blocco viene inviato di nuovo; i byte dei blocchi non vengono stampati), **TX fine file**, e alla fine **File** *file inviato: DIMENSIONE in TEMPO*, oppure *invio file fallito dopo X di Y: motivo*, oppure *invio file fermato dopo X di Y*. I motivi di errore sono scritti in inglese.

11.2 Opzioni di invio

La finestra **Opzioni di invio** ha sette schede.

Blocchi

- **Byte per blocco:** da 1 a 1048576 (predefinito 64). *Il file viene inviato in blocchi di questa dimensione; l'ultimo blocco contiene i byte rimanenti e può essere più piccolo.*
- **Ogni blocco, poi pausa** N **ms** (0 - 600000; 0 = alla massima velocità della porta): modalità con pausa.
- **Burst: dopo ogni blocco attendi la conferma (scheda Conferma)**: modalità con conferma.
- **Timeout risposta (ms):** (1 - 600000, predefinito 1000) e **Tentativi:** (0 - 1000, predefinito 3): quanto può durare l'attesa di una risposta e quante volte il blocco (o il prefisso, o il comando di recupero) viene inviato di nuovo senza risposta. *Se è andata persa solo la conferma, il ricevitore riceve quel blocco due volte* (il recupero, 11.6, lo evita).
- La stima, per esempio **12 blocco/i a 115200 baud: almeno 0.07 s (tempi di risposta dell'altra parte esclusi)**. Conta ogni byte sulla linea (dati, prefisso, marcatore di fine, le risposte attese in modalità burst) alla velocità della porta, più le pause. Senza file: **Seleziona un file per vedere il tempo stimato.**

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Prefisso iniziale - **Invia questo prima del file (segnala che segue un file)**: un messaggio inviato una volta prima del primo blocco.

Risposta al prefisso - **Attendi questa risposta prima di inviare il file**: la risposta attesa dall'altra parte dopo il prefisso. *Attesa dall'altra parte; anche valori incrementali e checksum vengono confrontati.* Se non arriva in tempo, il prefisso viene inviato di nuovo (Tentativi).

Recupero - **Consenti il recupero: continua un file interrotto da dove il ricevitore si è fermato** (11.6).

Risposta al recupero - attesa dopo il comando di recupero, seguita da 4 byte (big-endian): quanti byte del file il ricevitore ha già.

Conferma - modalità burst: la risposta attesa dopo ogni blocco prima di inviare il successivo. Un valore incrementale avanza del suo passo a ogni conferma.

Marcatore di fine - **Invia questo dopo l'ultimo blocco**.

Ogni scheda di comando o risposta ha **Usa il valore consigliato: ...**: spuntandolo l'editor viene riempito con il valore consigliato e bloccato. I valori consigliati sono comandi di 8 byte molto improbabili all'interno di un file, uguali sui due lati:

Comando	Valore consigliato
Prefisso iniziale	A5 5A 02 53 50 4D 46 01
Risposta al prefisso	A5 5A 06 53 50 4D 46 01
Recupero	A5 5A 02 53 50 4D 46 12
Risposta al recupero	A5 5A 06 53 50 4D 46 12 (+ 4 byte)
Conferma	A5 06 + CRC-16/MODBUS del blocco
Marcatore di fine	A5 5A 04 53 50 4D 46 04

La conferma consigliata contiene un CRC-16/MODBUS del blocco che conferma (il checksum *include il messaggio a cui risponde*), quindi è anche un controllo di integrità di ogni blocco.

OK controlla le impostazioni: ogni comando in uso deve essere valido e non vuoto (*Prefisso iniziale: lo schema è vuoto.*), e i comandi inviati - come le risposte attese - devono essere diversi e nessuno deve contenere un altro (**"Prefisso iniziale" e "Marcatore di fine" devono essere diversi e nessuno dei due può contenere l'altro: l'altra parte potrebbe scambiarli.**). Viene mostrata la scheda con il problema.

11.3 Le due modalità di invio

- **Con pausa** (*Ogni blocco, poi pausa*): blocco, pausa, blocco... Il ricevitore deve tenere il passo da solo. Usare una pausa abbastanza lunga per un dispositivo che scrive ogni blocco in memoria.
- **Burst** (*dopo ogni blocco attendi la conferma*): blocco, attesa della conferma, blocco successivo. Senza conferma entro il timeout di risposta lo stesso blocco viene inviato di nuovo, fino a Tentativi volte; poi il trasferimento fallisce con un motivo che indica quale risposta era attesa, per esempio *no confirmation for chunk 4 (expected: A5 06 ...)*.

11.4 Catturare i file

Cattura file... apre le opzioni **Cattura file** (11.5); **OK** attiva la cattura. Il pulsante diventa **Ferma cattura**, il testo di stato indica **in attesa di un file...** e la console mostra **File** *cattura file in PERCORSO*. La cattura si può attivare con la porta chiusa: parte all'apertura della porta.

- I byte ricevuti prima del prefisso iniziale vengono mostrati e trattati come normale RX (risposte automatiche comprese).

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- Quando arriva il prefisso iniziale inizia un file: la console mostra *inizio file riconosciuto: PERCORSO viene riscritto dall'inizio* e il testo di stato mostra **ricezione: DIMENSIONE**. Da quel momento i byte sono dati del file: vengono scritti nel file di uscita e non mostrati nella console.
- Le risposte inviate dalla cattura compaiono come **TX risposta file**.
- Il file termina con il marcatore di fine o dopo il tempo di silenzio; la console mostra *file ricevuto: DIMENSIONE in PERCORSO* e lo stato **N file ricevuti - in attesa di un file...**. La cattura attende poi il file successivo.
- **Ferma cattura** la disattiva (*cattura file fermata*).

Opzioni di cattura... modifica le opzioni anche durante la cattura: un file in ricezione termina con le opzioni precedenti, il successivo usa le nuove (la console indica *opzioni di cattura modificate (un file in ricezione termina con le precedenti); scrittura su PERCORSO*).

11.5 Opzioni di cattura

File

- **Scrivi i dati del file ricevuto in:** il file di uscita, con **Sfoglia...**. *Vengono scritti solo i dati del file (niente prefisso, risposte o marcatore di fine). Un nuovo file sostituisce il contenuto del file di uscita; un file recuperato (scheda Recupero) lo continua.*
- **Byte per gruppo (risposta a ciascuno):** da 1 a 1048576 (predefinito 64).
- **Il file termina dopo un silenzio di (ms, 0 = mai):** da 0 a 3600000 (predefinito 2000).

Prefisso iniziale - *Un file inizia quando questo viene ricevuto. I byte precedenti sono mostrati come RX normale.* Obbligatorio.

Risposta al prefisso - **Rispondi al prefisso con questo**.

Recupero - **Accetta il recupero: un file interrotto continua (il file di uscita viene mantenuto)**.

Risposta al recupero - *Inviata dopo il comando di recupero, seguita dalla dimensione del file di uscita (4 byte, big-endian).*

Risposta ai gruppi - **Rispondi a ogni gruppo di byte con questo**: *Inviata dopo ogni gruppo; un ultimo gruppo più piccolo riceve risposta quando la linea tace. Un valore incrementale avanza a ogni risposta. Se il suo checksum include il gruppo, usa come dimensione del gruppo quella dei blocchi del mittente, così ogni risposta copre esattamente un blocco.*

Marcatore di fine - **Il file termina con questo marcatore**: *Il marcatore non viene scritto nel file.*

OK rifiuta un file di uscita vuoto (**Scegli il file in cui scrivere i dati.**), un file che non finirebbe mai (**Un file non finirebbe mai: usa un marcatore di fine o un tempo di silenzio.**) e comandi in conflitto, come per l'invio.

Per trasferire file tra due installazioni di SerialPortManager usare gli stessi comandi sui due lati: spuntare **Usa il valore consigliato** ovunque, impostare il **Byte per blocco** del mittente uguale al **Byte per gruppo** della cattura e scegliere la modalità burst sul mittente con la risposta ai gruppi sulla cattura.

11.6 Recupero di un trasferimento interrotto

Quando un invio si ferma a metà (Ferma, una conferma mancata, la porta chiusa, il programma chiuso), il programma ricorda il file e fin dove è arrivato. Se il recupero è consentito nelle opzioni di invio e si invia di nuovo lo stesso file (stesso percorso e dimensione), chiede **Recuperare il file?**: *L'ultimo invio di NOME si è fermato dopo X di Y (confermati).*

- **Recupera**: al posto del prefisso iniziale viene inviato il comando di recupero; il ricevitore mantiene quello che ha e risponde con la risposta al recupero seguita dalla dimensione del suo file (4 byte,

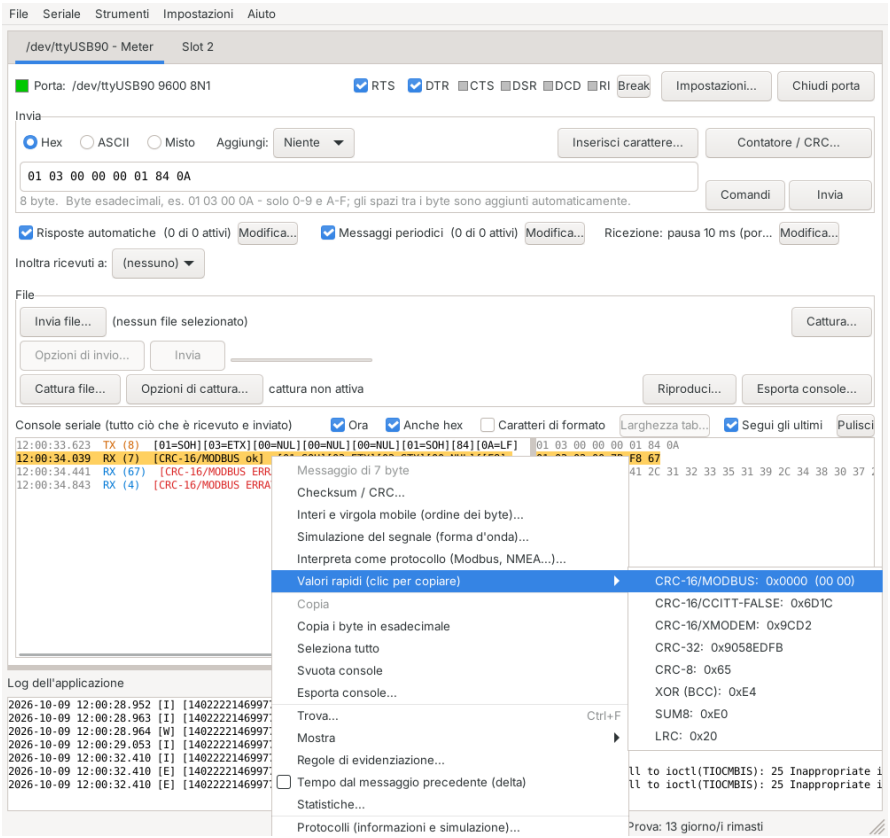
Copyright © 2026 Fation Coga. Tutti i diritti riservati.

big-endian); l'invio continua da quel byte. La console mostra **TX recupero file** e *il ricevitore ha X di Y: si continua da lì*. Poiché la dimensione del ricevitore è quella che conta, un blocco la cui conferma è andata persa non viene né inviato due volte né saltato.

- **Ricomincia**: il file viene inviato dall'inizio e il ricevitore lo ricomincia.
- **Annulla**: non viene inviato nulla.

Dal lato della cattura, **Accetta il recupero** mantiene il file di uscita quando arriva il comando di recupero - al posto del prefisso iniziale, oppure a metà di un file non terminato - risponde con la dimensione del file e aggiunge in fondo i dati che seguono (*recupero: X già in PERCORSO, si continua*). Un prefisso iniziale o un comando di recupero ripetuto prima dell'arrivo di qualsiasi dato (perché il mittente non ha ricevuto la risposta) riceve di nuovo risposta invece di essere preso come dato del file.

12. Clic destro sulla console: analisi di un messaggio



Il menu del clic destro della console

Fare clic destro su una riga di messaggio della console per analizzarne i byte. Si può anche selezionare prima una parte di un messaggio - nel riquadro di testo o in quello esadecimale, anche su più righe - e fare clic destro dentro la selezione: allora vengono usati solo i byte selezionati (le righe di uno stesso messaggio formano un'unica parte di esso; i byte di più messaggi vengono uniti in ordine). Con la tastiera (tasto menu contestuale) il menu usa la selezione.

Il menu contiene:

- un'intestazione (non selezionabile): **Messaggio di N byte** per un messaggio intero, oppure **N byte selezionati** per una selezione;
- **Checksum / CRC...** - apre un Analizzatore di byte sulla pagina dei checksum;
- **Interi e virgola mobile (ordine dei byte)...** - Analizzatore di byte, pagina dei numeri;

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- **Simulazione del segnale (forma d'onda)...** - Analizzatore di byte, pagina del segnale, partendo da velocità, bit di dati, parità e bit di stop di questa porta;
- **Interpreta come protocollo (Modbus, NMEA...)...** - Analizzatore di byte, pagina del protocollo;
- **Valori rapidi (clic per copiare)** - un sottomenu con i risultati più usati; facendo clic su uno il suo valore viene copiato negli appunti:
  - CRC-16/MODBUS con i suoi due byte come vengono inviati, prima il byte basso, per esempio `0x0A84 (84 0A)`;
  - CRC-16/CCITT-FALSE, CRC-16/XMODEM, CRC-32, CRC-8, XOR (BCC), SUM8, LRC;
  - per esattamente 2 byte: UInt16, Int16 e Float16 in BE e LE; per 4 byte: UInt32, Int32 e Float32 in BE, LE e CDAB; per 8 byte: UInt64, Int64 e Float64 in BE, LE e CDAB;
- **Copia** - copia il testo selezionato della console;
- **Copia i byte in esadecimale** - copia i byte del messaggio o della selezione, per esempio `01 03 00 00`;
- **Seleziona tutto**, **Svuota console**;
- **Esporta console...** - 8.8;
- **Trova...** (Ctrl-F), **Mostra** (ricevuti / inviati / note), **Regole di evidenziazione...** - 8.5;
- **Tempo dal messaggio precedente (delta)** - 8.6; **Statistiche...** - 8.7;
- **Protocolli (informazioni e simulazione)...** - apre la finestra Protocolli.

Le righe senza byte (note, righe dei blocchi dei file) lasciano disattivate le voci di analisi. Gli analizzatori aperti dalla console hanno come titolo l'origine, per esempio *Analizzatore di byte - RX 14:03:21.512 - COM3 - Contatore*, oppure *selezione di ...* per una selezione.

13. L'Analizzatore di byte

L'Analizzatore di byte è una finestra indipendente per analizzare una sequenza di byte. Si apre:

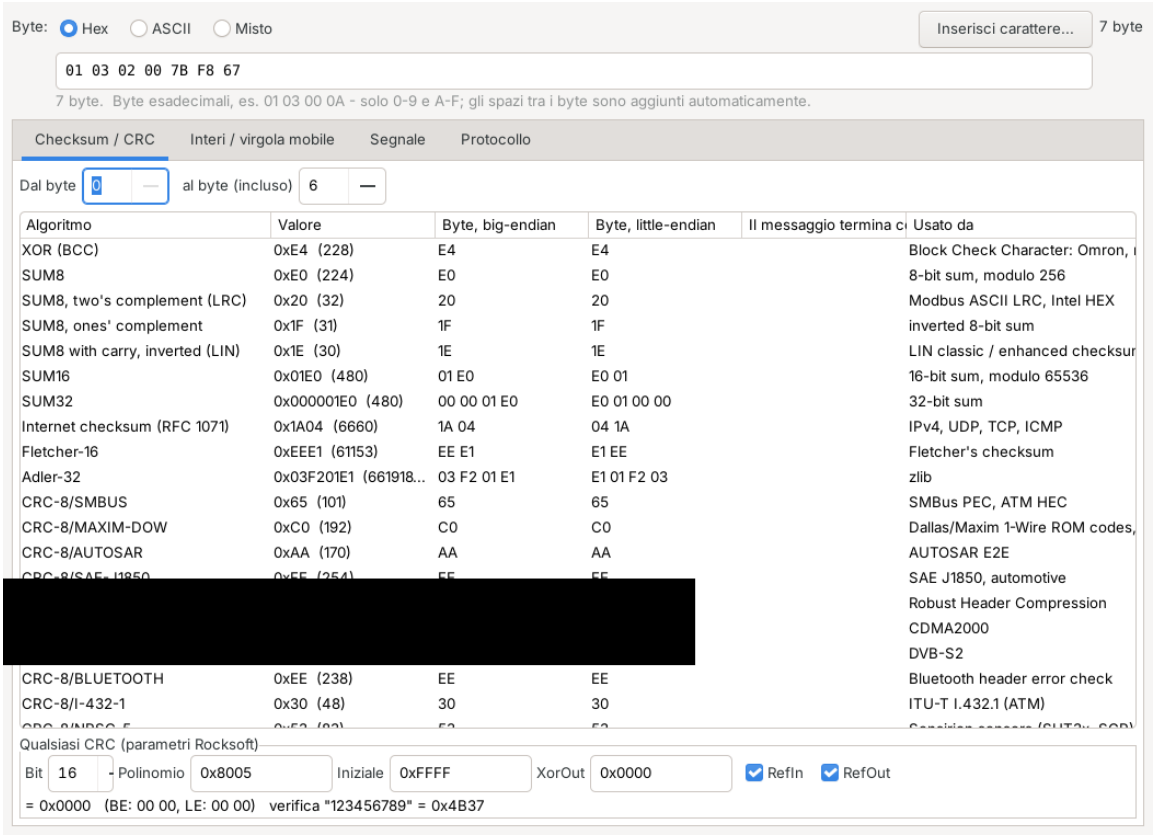
- dal menu del clic destro della console (capitolo 12), sui byte di un messaggio o di una selezione, alla pagina scelta;
- con **Strumenti > Analizzatore di byte...** (Ctrl-B), sui byte della casella di invio della scheda attuale (vuoto se il testo della casella non è valido).

Si possono tenere aperti più analizzatori contemporaneamente, ognuno con i propri byte; restano aperti mentre si continua a lavorare nella finestra principale.

In alto, **Byte:** mostra i byte in un campo dati modificabile (Hex, ASCII o Misto, con **Inserisci carattere...**) e il loro numero (**N byte**). Ogni modifica viene analizzata subito su tutte le pagine. Le quattro pagine sono **Checksum / CRC**, **Interi / virgola mobile**, **Segnale** e **Protocollo**.

13.1 Checksum / CRC

Copyright © 2026 Fation Coga. Tutti i diritti riservati.



La pagina dei checksum dell'Analizzatore di byte

- **Dal byte** / **al byte (incluso)**: l'intervallo su cui si calcolano i checksum. I byte sono numerati da 0. La fine segue i dati: se è sull'ultimo byte, vi resta quando si aggiungono byte.
- L'elenco mostra circa 50 algoritmi (52 in questa versione): 10 somme e checksum e 42 CRC da CRC-8 a CRC-64, ognuno con i suoi parametri pubblicati (Rocksoft). Colonne:
  - **Algoritmo** - per esempio CRC-16/MODBUS, CRC-32/ISO-HDLC, XOR (BCC);
  - **Valore** - esadecimale e decimale;
  - **Byte, big-endian** e **Byte, little-endian** - i byte da aggiungere al messaggio nei due ordini;
  - **Il messaggio termina con esso** - **sì (BE)** o **sì (LE)**, in verde, quando gli ultimi byte del messaggio sono già questo checksum dei byte che li precedono (dal primo byte dell'intervallo): così si scopre quale checksum usa un dispositivo;
  - **Usato da** - dove si usa l'algoritmo (per esempio *Modbus RTU (sent low byte first)*, *Dallas/Maxim 1-Wire ROM codes, DS18B20*, *Ethernet, ZIP, PNG*; questa colonna è in inglese).
- Clic destro sull'elenco: **Copia le righe selezionate** o **Copia tutte le righe** (separate da tabulazioni, pronte per un foglio di calcolo).

Le somme e i checksum: XOR (BCC), SUM8, SUM8 complemento a due (LRC), SUM8 complemento a uno, SUM8 con riporto invertito (LIN), SUM16, SUM32, checksum di Internet (RFC 1071), Fletcher-16, Adler-32. I CRC: dieci varianti CRC-8 (SMBUS, MAXIM-DOW, AUTOSAR, SAE-J1850, ROHC, CDMA2000, DVB-S2, BLUETOOTH, I-432-1, NRSC-5), CRC-15/CAN, diciannove varianti CRC-16 (MODBUS, ARC, IBM-3740, XMODEM, KERMIT, IBM-SDLC, MCRF4XX, GENIBUS, SPI-FUJITSU, USB, MAXIM-DOW, UMTS, CMS, DDS-110, DNP, EN-13757, PROFIBUS, T10-DIF, TELEDISK), CRC-24/OPENPGP, CRC-24/BLE, sette varianti CRC-32 (ISO-HDLC, ISCSI, BZIP2, MPEG-2, CKSUM, JAMCRC, AUTOSAR) e tre CRC-64 (ECMA-182, XZ, GO-ISO).

Qualsiasi CRC (parametri Rocksoft) calcola un CRC che non è nell'elenco: **Bit** (larghezza, 1 - 64), **Polinomio**, **Iniziale**, **XorOut** (numeri come 0x1021, 4129 o 0b...), **RefIn** e **RefOut**. Parte con i parametri di

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

CRC-16/MODBUS. La riga del risultato mostra il valore, i suoi byte big-endian e little-endian e il valore di *verifica* - il CRC del testo ASCII "123456789" - da confrontare con il valore di verifica pubblicato per l'algoritmo per confermare i parametri. La riga si legge = *VALORE (BE: byte, LE: byte) verifica "123456789" = VERIFICA*; con i parametri iniziali di CRC-16/MODBUS la verifica è 0x4B37. Parametri non validi: **Scrivi Polinomio, Iniziale e XorOut come numeri (0x1021, 4129, 0b...).**

13.2 Interi / virgola mobile

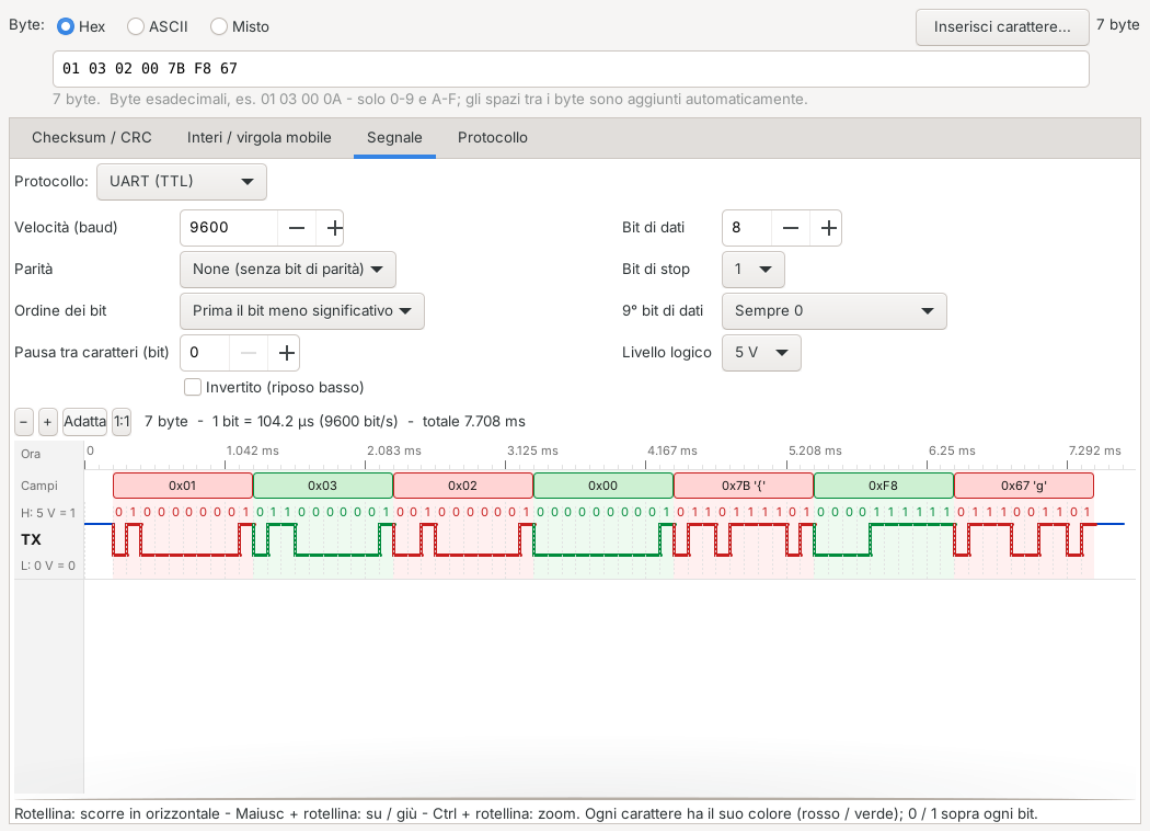
- **Posizione**: il primo byte da leggere (0 = primo byte).
- **Ordine dei byte**: come sono disposti sulla linea i byte di un numero. Con i byte A B C D nell'ordine di arrivo:
  - **ABCD - big-endian**: prima il byte più significativo (registri Modbus, la maggior parte dei protocolli di rete);
  - **DCBA - little-endian**: prima il byte meno significativo (PC e la maggior parte dei microcontrollori, in memoria);
  - **BADC - big-endian, byte scambiati**: i due byte di ogni word a 16 bit scambiati;
  - **CDAB - little-endian, word scambiate**: le word a 16 bit in ordine inverso, ognuna big-endian (valori a 32 bit su due registri Modbus, prima la word bassa). Per i valori a 2 byte CDAB equivale ad ABCD.
- **Scala x**: il valore viene mostrato anche moltiplicato per questo, per esempio 0.1 per una lettura in decimi.

L'elenco mostra ogni tipo a quella posizione: Int8, UInt8, Int16, UInt16, Int24, UInt24, Int32, UInt32, Int64, UInt64, Float16 (IEEE 754 half), Float32 (IEEE 754 single), Float64 (IEEE 754 double), BCD 2 byte, BCD 4 byte. Colonne: **Tipo**, **Valore**, **Scalato** (quando la scala non è 1), **Esadecimale (big-endian)** (i byte riordinati in big-endian) e **Byte sulla linea** (i byte come sono nel messaggio). Un tipo che non ci sta indica **servono N byte dalla posizione**; un valore BCD con una cifra oltre 9 indica **non BCD (una cifra oltre 9)**. Clic destro per copiare le righe.

Valore in byte (e dove si trovano nel messaggio) fa il contrario: scegliere un tipo (all'inizio Float32), scrivere un valore (decimale, 0x esadecimale, 0b binario; virgola mobile come 1.5 o 1e-3) e la riga mostra i byte nell'ordine scelto sopra, per esempio **Byte: 3F C0 00 00**, seguiti da **trovati alla posizione 4** (in verde) oppure **non presenti nel messaggio**. Un valore non valido mostra **Non valido** con il motivo (vuoto, non è un intero, fuori intervallo, negativo, non è un numero, troppe cifre, solo cifre 0-9).

13.3 Segnale

Copyright © 2026 Fation Coga. Tutti i diritti riservati.



La pagina Segnale: i byte di un messaggio come forma d'onda UART

La pagina Segnale disegna la forma d'onda che i byte producono sui fili. **Protocollo:** sceglie UART (TTL), RS-232, RS-422, RS-485, SPI, I2C, 1-Wire, CAN, LIN, JTAG o NRZ / Manchester; sotto ci sono i parametri di quel protocollo (gli stessi della finestra Protocolli, capitolo 14.3). I byte sono quelli in alto nell'analizzatore.

Per UART (TTL), RS-232, RS-422 e RS-485 l'analizzatore parte dalle impostazioni della porta da cui vengono i byte: velocità, bit di dati, parità e bit di stop. Così la forma d'onda mostra esattamente i byte del messaggio come hanno viaggiato sul filo. Il protocollo scelto e gli altri parametri vengono ricordati. La vista della forma d'onda è descritta nel capitolo 14.4.

13.4 Protocollo

Interpreta come sceglie il decodificatore:

- **Automatico**: Modbus ASCII se i dati iniziano con `:` e sono validi, NMEA 0183 se iniziano con `$` o `!` e sono validi, altrimenti Modbus RTU se il suo CRC è corretto, altrimenti byte uno per uno;
- **Modbus RTU**: indirizzo, funzione, campi dati, CRC-16/MODBUS (prima il byte basso);
- **Modbus ASCII**: inizio `:`, indirizzo, funzione, dati, LRC, fine CR LF;
- **NMEA 0183**: inizio `$` o `!`, talker e frase, ogni campo (con i nomi per le frasi GGA e RMC), checksum (XOR), fine CR LF;
- **Byte uno per uno**: ogni byte in esadecimale, decimale, binario e come carattere.

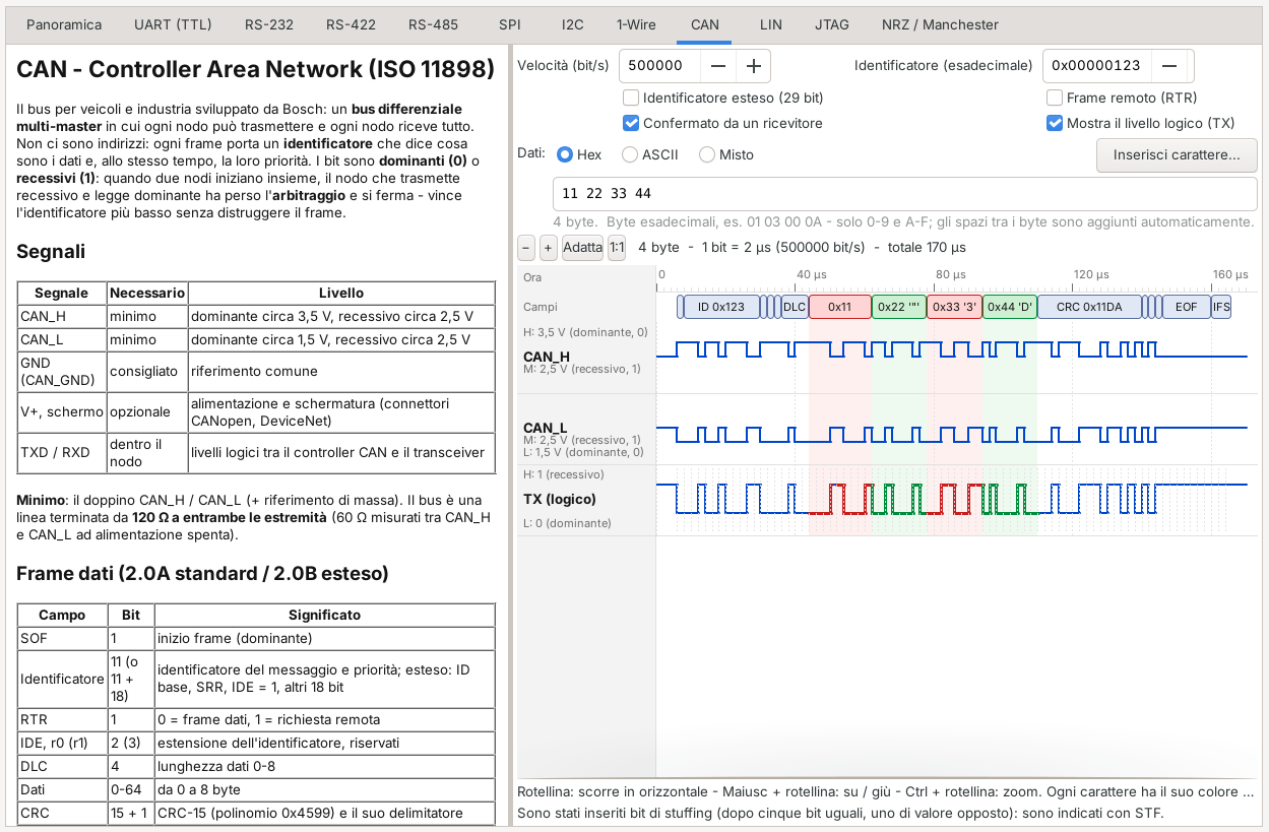
Una riga di riepilogo accanto alla scelta (verde se il frame è valido, rossa se non lo è) è seguita dall'elenco **Campo**, **Byte** (l'intervallo di byte nel messaggio, per esempio 2-3) e **Valore**. I controlli corretti sono verdi, quelli errati rossi, con il valore atteso: per esempio *CRC-16/MODBUS (prima il byte basso): 0x0A84 correct*, oppure *wrong, expected 0x0A84 = 84 0A*. Il riepilogo e i valori sono scritti in inglese.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Per Modbus il decodificatore conosce i codici funzione comuni (01 Read Coils, 02 Read Discrete Inputs, 03 Read Holding Registers, 04 Read Input Registers, 05 Write Single Coil, 06 Write Single Register, 0F Write Multiple Coils, 10 Write Multiple Registers e altri), distingue richieste e risposte dalla loro lunghezza, mostra i registri in decimale, esadecimale e con segno, e mostra in rosso le risposte di eccezione (codice funzione + 0x80) con il loro **Codice di eccezione**.

14. La finestra Protocolli

Strumenti > Protocolli (informazioni e simulazione)... (Ctrl-P), oppure **Protocolli (informazioni e simulazione)...** nel menu del clic destro della console, apre la finestra **Protocolli di comunicazione standard - informazioni e simulazione**. È indipendente dalla finestra principale; riaprendola viene portata in primo piano la stessa finestra.



La finestra Protocolli: la scheda CAN, spiegazione e simulazione

14.1 La scheda Panoramica

La scheda **Panoramica** (*I protocolli di comunicazione in breve*) spiega che cosa distingue i protocolli (asincroni o sincroni, quanti dispositivi condividono i fili, velocità, distanza, controllo degli errori) e li confronta in una tabella:

Protocollo	Clock	Dispositivi	Velocità tipica	Distanza
UART (TTL)	asincrono	2	9600 - 115200 bit/s	1-2 m
RS-232	asincrono	2	fino a 20 kbit/s	15 m
RS-422	asincrono	1 driver, 10 ricevitori	fino a 10 Mbit/s	1200 m
RS-485	asincrono	32 carichi unitari	fino a 10 Mbit/s	1200 m
SPI	sincrono	1 master, più slave	1 - 50 MHz	sulla scheda
I²C	sincrono	112 indirizzi (7 bit)	100 kbit/s - 5 Mbit/s	< 1 m
1-Wire	slot temporali	molti	15,4 / 125 kbit/s	decine di metri

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Protocollo	Clock	Dispositivi	Velocità tipica	Distanza
CAN 2.0	asincrono	~110 nodi	10 kbit/s - 1 Mbit/s	40 m - 1 km
LIN	asincrono	fino a 16 nodi	1 - 20 kbit/s	40 m
JTAG	sincrono (TCK)	una catena di chip	1 - 50 MHz	sulla scheda

La tabella della finestra indica anche i fili di segnale (minimo - massimo), il tipo di duplex e il controllo degli errori di ogni protocollo, e la scheda termina con le istruzioni per usare la finestra.

14.2 Le schede dei protocolli

C'è una scheda per protocollo: **UART (TTL)**, **RS-232**, **RS-422**, **RS-485**, **SPI**, **I2C**, **1-Wire**, **CAN**, **LIN** e **NRZ / Manchester** (codifiche di linea). Ogni scheda ha due parti, separate da una barra trascinabile:

- **A sinistra: la spiegazione** - a che cosa serve il protocollo; i segnali necessari al minimo e al massimo, con direzione e significato; livelli di tensione; struttura del frame; velocità e distanze; controllo degli errori; dispositivi tipici e significato dei dati di esempio.
- **A destra: il simulatore** - **Dati:** (un campo dati modificabile, Hex, ASCII o Misto), ogni parametro del protocollo, la barra dello zoom, la forma d'onda dal vivo, la riga che descrive cosa c'è sotto il mouse, e le note.

Ogni scheda parte con dati di esempio: UART `48 69` ("Hi"), RS-232 `4F 4B 0D` ("OK" + CR), RS-422 `55 AA`, RS-485 `01 03 00 00 00 01 84 0A` (Modbus RTU: lettura di un holding register dello slave 1), SPI `9F 00 00 00` (comando JEDEC "read identification" di una flash SPI), I²C `10 A5`, 1-Wire `CC 44` (Skip ROM, Convert T di un DS18B20), CAN `11 22 33 44`, LIN `55 93 E5`, codifiche di linea `4D 3C`. Modificando i dati o un parametro la forma d'onda si aggiorna subito. I parametri vengono ricordati tra una sessione e l'altra.

14.3 Parametri del simulatore

I parametri esadecimali (indirizzi, identificatori) si mostrano e si scrivono in esadecimale. Passando con il mouse su un parametro compare un suggerimento, dove previsto.

UART (TTL)

Parametro	Valori (predefinito)
Velocità (baud)	50 - 12000000 (9600)
Bit di dati	5 - 9 (8)
Parità	None (senza bit di parità), Dispari, Pari, Mark (sempre 1), Space (sempre 0)
Bit di stop	1, 1.5, 2
Ordine dei bit	Prima il bit meno significativo, Prima il bit più significativo
9° bit di dati	Sempre 0, Sempre 1, 1 sul primo byte (indirizzo) - con 9 bit di dati il 9° bit indica un indirizzo (modo multiprocessore / multidrop)
Pausa tra caratteri (bit)	0 - 50 (0 = uno dopo l'altro)
Livello logico	5 V, 3.3 V, 1.8 V
Invertito (riposo basso)	riposo basso e 1 = 0 V, come forniscono alcuni optoisolatori e traslatori di livello

RS-232 e **RS-422**: i primi sette parametri UART (da Velocità (baud) a Pausa tra caratteri). **RS-485**: gli stessi più **Mostra DE (abilitazione driver)** (attivo): l'abilitazione del driver del transceiver, alta mentre questo nodo pilota il bus (half duplex).

SPI

Parametro	Valori (predefinito)
Clock (Hz)	1000 - 200000000 (1000000)
Modo	da Modo 0 (CPOL 0, CPHA 0) a Modo 3 (CPOL 1, CPHA 1)

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Parametro	Valori (predefinito)
Bit per parola	8, 16, 32
Ordine dei bit	Prima il bit più significativo, Prima il bit meno significativo
Linee dati	1 - standard (MOSI / MISO), 2 - dual (IO0-IO1), 4 - quad (IO0-IO3)
Risposta su MISO	Niente (alta impedenza), 0xFF (slave inattivo), La parola precedente (registro a scorrimento), Uguale a MOSI (loopback)
Slave (linee CS)	1 - 8 (1)
Slave selezionato	1 - 8 (1)
CS rilasciato tra le parole	disattivo - come richiedono alcuni ADC e DAC
CS attivo alto	disattivo (chip select attivo basso)

I2C

Parametro	Valori (predefinito)
Velocità	Standard-mode, 100 kbit/s; Fast-mode, 400 kbit/s; Fast-mode Plus, 1 Mbit/s; High-speed mode, 3,4 Mbit/s; Ultra Fast-mode, 5 Mbit/s (solo scrittura)
Indirizzo (esadecimale)	0 - 3FF (50)
Indirizzo a 10 bit	disattivo
Direzione	Scrittura (dal master allo slave), Lettura (dallo slave al master)
Byte del registro	0 - 2 (0): registro scritto per primo; in lettura segue uno start ripetuto
Registro (esadecimale)	0 - FFFF (0)
Lo slave risponde ACK	attivo; disattivo = nessuno risponde all'indirizzo (NACK) e il master si ferma
NACK dopo l'ultimo byte letto	attivo

1-Wire: **Velocità** (Standard (~15,4 kbit/s), Overdrive (~125 kbit/s)), **Prima l'impulso di reset** (attivo), **Un dispositivo risponde (presenza)** (attivo), **Chi trasmette** (Il master scrive, Lo slave risponde (slot di lettura)).

CAN

Parametro	Valori (predefinito)
Velocità (bit/s)	10000 - 1000000 (500000)
Identificatore (esadecimale)	0 - 1FFFFFFF (123)
Identificatore esteso (29 bit)	disattivo
Frame remoto (RTR)	disattivo: una richiesta di dati, campo dati vuoto, DLC indica quanti byte sono richiesti
Confermato da un ricevitore	attivo; disattivo = lo slot ACK resta recessivo (un errore)
Mostra il livello logico (TX)	attivo

LIN: **Velocità (baud)** (1000 - 20000, 19200), **ID del frame (esadecimale, 0-3F)** (10), **Checksum avanzato (LIN 2.x)** (attivo; il checksum copre anche l'identificatore protetto, non per i frame diagnostici 3C / 3D), **Durata del break (bit)** (13 - 26, 13), **Con risposta (dati + checksum)** (attivo; disattivo = solo l'intestazione, la richiesta del master).

JTAG: **Clock (Hz)** (1000 - 100000000, 1000000), **Scansione** (Solo registro dati (scansione DR), Istruzione, poi dati (scansione IR + DR); predefinita la seconda), **Bit del registro istruzioni** (2 - 32, 4), **Istruzione (esadecimale)** (0xE), **Prima il reset della TAP (TMS = 1 x 5)** (attivo), **Risposta su TDO** (BYPASS: TDI un clock dopo, Tutto a uno, Uguale a TDI (loopback)), **Mostra TRST* (linea di reset opzionale)** (disattivo). Le righe sono TRST* (opzionale), TCK, TMS, TDI e TDO (tratteggiata in alta impedenza); la riga Campi indica lo stato TAP di ogni clock (Test-Logic-Reset, Run-Test/Idle, Select-DR-Scan, Capture-IR, Shift-IR, Update-IR...), poi l'istruzione e ogni byte di dati, fatti scorrere dal bit meno significativo.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

NRZ / Manchester: **Velocità (bit/s)** (1 - 100000000, 1000), **Codifica di linea** (NRZ-L, NRZ-I (cambia sull'1), Manchester IEEE 802.3 (1 = da basso ad alto), Manchester G.E. Thomas (1 = da alto a basso), Manchester differenziale (cambia sullo 0), RZ (ritorno a zero), AMI bipolare; predefinito Manchester IEEE 802.3), **Ordine dei bit** (Prima il bit più significativo, Prima il bit meno significativo), **Mostra il clock** (attivo).

14.4 La vista della forma d'onda

La stessa vista è usata nella finestra Protocolli e nella pagina Segnale dell'Analizzatore di byte.

- **La barra sopra la vista**: il pulsante meno (**Riduci (Ctrl + rotellina)**), **+** (**Ingrandisci (Ctrl + rotellina)**), **Adatta** (**Mostra tutta la trasmissione**) e **1:1** (**Zoom normale: ogni bit leggibile**), seguiti dal riepilogo, per esempio *2 byte - 1 bit = 104.2 µs (9600 bit/s) - totale 2.5 ms*.
- **Ora**: il righello dei tempi in alto, con una tacca per ogni bit e un'etichetta di tempo dove c'è spazio.
- **Campi**: un riquadro per carattere, alternativamente rosso e verde, con il suo valore (per esempio *0x48 'H'*), e i campi del protocollo in blu (per esempio *SOF*, *ID 0x123*, *DLC 4*, *CRC 0x1A2B*, *Address 0x50 W*, *Sync 0x55*). Dietro i fili, bande dello stesso rosso e verde mostrano dove ogni carattere inizia e finisce.
- **Una riga per filo**, con il nome a sinistra (per esempio TX, TxD, TX+ (Y), B (+), SCLK, MOSI, MISO, CS, SDA, SCL, DQ, CAN_H, CAN_L, LIN, Clock). Sotto il nome, **H:**, **L:** e, dove usato, **M:** indicano che cosa significano il livello alto, basso e intermedio su quel filo (per esempio *H: 3,5 V (dominante, 0)*, *M: 2,5 V (recessivo, 1)*). Il livello intermedio si usa per il CAN recessivo, lo zero dell'AMI e - tratteggiato - per una linea rilasciata o non pilotata (alta impedenza).
- Sopra ogni bit è scritto il suo valore **0** o **1** nel colore del suo carattere; un bit di stuffing è scritto come `0*` o `1*`.
- Sotto la traccia, il ruolo dei bit di protocollo:

Sigla	Significato
ST	bit di start (UART), condizione di START (I²C), SOF (CAN)
SP	bit di stop (UART), condizione di STOP (I²C)
PAR	bit di parità
ACK / NAK	conferma / mancata conferma (NAK in rosso)
STF	bit di stuffing (CAN)
R/W	bit lettura / scrittura (I²C)
BRK	break (LIN)
DEL	delimitatore (CAN, LIN)
Sr	start ripetuto (I²C)
RST, PD	impulso di reset, impulso di presenza (1-Wire)
ID, CTL, DLC, CRC	identificatore, bit di controllo, lunghezza dati, CRC
EOF, IFS	fine del frame, spazio tra i frame (CAN)

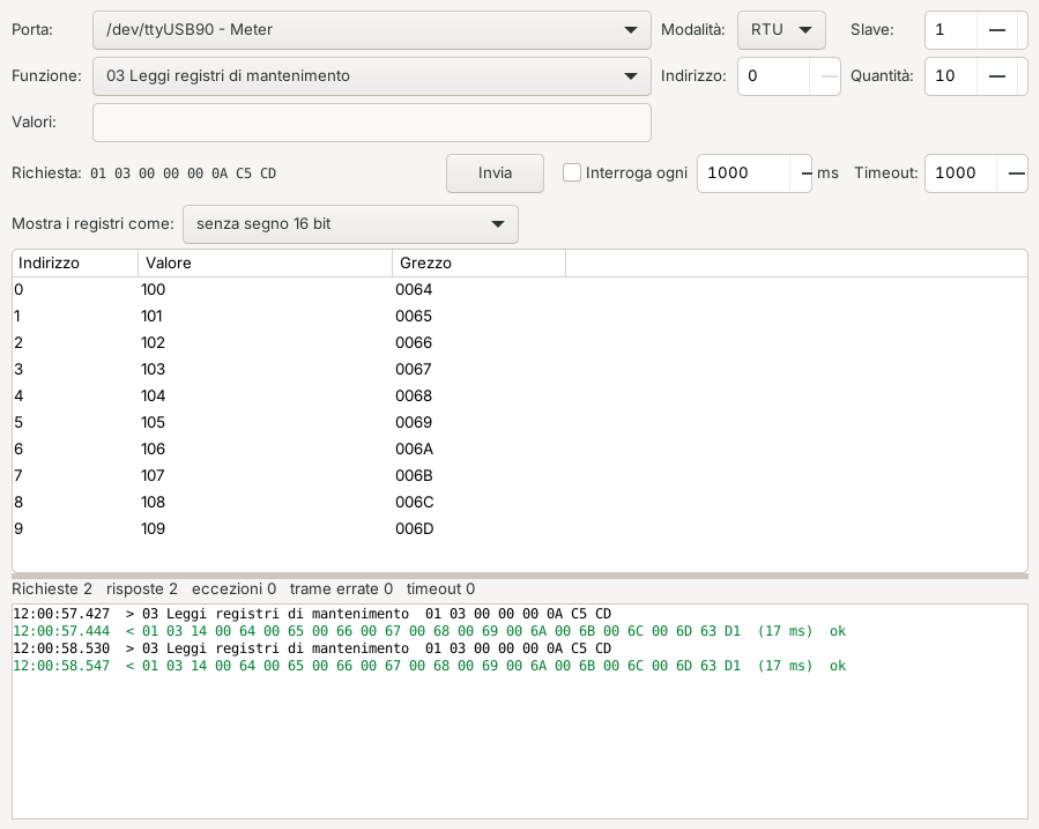
Spostarsi. La rotellina del mouse scorre in orizzontale; Maiusc + rotellina scorre in su e in giù; Ctrl + rotellina ingrandisce attorno al mouse (il tempo sotto il mouse resta sotto il mouse). Funzionano anche le barre di scorrimento.

Leggere. La riga sotto la vista descrive il punto sotto il mouse: tempo, campo, filo, livello e suo significato, il bit con il suo ruolo, la sua posizione nel byte e il suo inizio e fine - per esempio *t = 1.25 ms* | *0x48 'H'* | *TX: alto (5 V = 1)* | *Bit di dati 3 del byte 0 = 1 [1.146 ms - 1.25 ms]*. Senza il mouse sulla vista mostra il suggerimento **Rotellina: scorre in orizzontale - Maiusc + rotellina: su / giù - Ctrl + rotellina: zoom. Ogni carattere ha il suo colore (rosso / verde); 0 / 1 sopra ogni bit.**

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Note. Sotto, alcune note spiegano che cosa ha fatto la simulazione con le impostazioni scelte, per esempio *CAN 2.0 porta al massimo 8 byte di dati: vengono inviati solo i primi 8 (CAN FD ne porta fino a 64).*, *Un frame LIN porta al massimo 8 byte di dati: vengono inviati solo i primi 8.*, *Un indirizzo a 7 bit arriva fino a 0x7F: i bit più alti vengono ignorati.*, *Lo slave non ha confermato: il master interrompe il trasferimento.*, *Nessun impulso di presenza: nessun dispositivo ha risposto al reset.*, *L'ultima parola è completata con byte a zero.*, *Sono stati inseriti bit di stuffing (dopo cinque bit uguali, uno di valore opposto): sono indicati con STF.*, *Alcuni byte non entrano nei bit di dati: vengono inviati solo i loro bit bassi.*

15. Il Master Modbus



La finestra Master Modbus che legge dieci registri di mantenimento

Strumenti > Master Modbus... (Ctrl-M) legge e scrive le bobine e i registri di uno slave Modbus RTU o ASCII attraverso la porta di una delle schede.

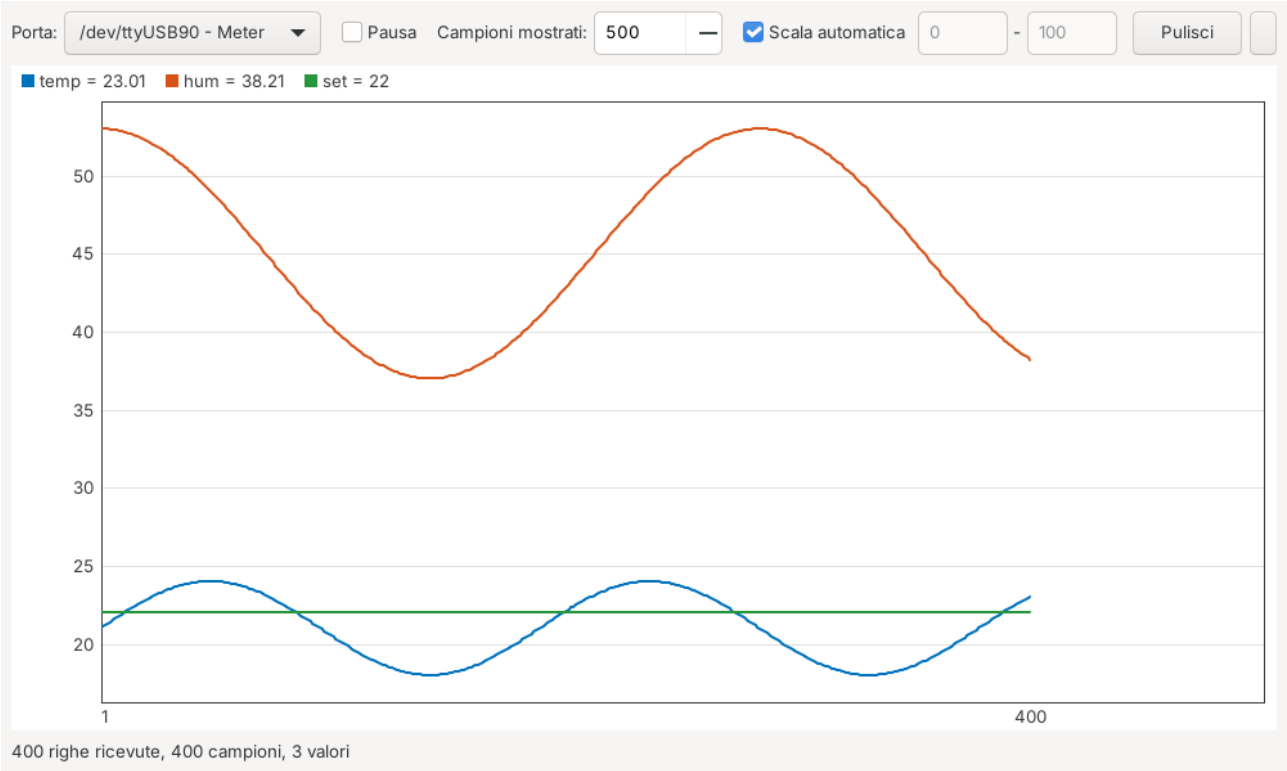
- **Porta:** la scheda attraverso la cui porta passano le richieste - deve essere aperta; la sua console mostra le richieste come **TX Modbus** e le risposte come **RX**.
- **Modalità: RTU** (binario, CRC-16) o **ASCII** (testo esadecimale con `:`, LRC, CR LF).
- **Slave:** 1 - 247; 0 = broadcast (solo scritture, nessuno risponde).
- **Funzione: 01 Leggi bobine**, **02 Leggi ingressi digitali**, **03 Leggi registri di mantenimento**, **04 Leggi registri di ingresso**, **05 Scrivi una bobina**, **06 Scrivi un registro**, **15 Scrivi più bobine**, **16 Scrivi più registri**.
- **Indirizzo:** l'indirizzo del protocollo, da 0 (il registro 40001 è 0); **Quantità:** 1 - 125 registri o 1 - 2000 bobine per le letture.
- **Valori:** per le scritture: numeri, decimali o esadecimali `0x`, separati da spazi (bobine: 0 / 1).
- **Richiesta:** la trama come verrà inviata, o in rosso perché non può esserlo.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Invia la invia una volta; **Interroga ogni N ms** la invia di continuo (la successiva dopo la risposta o il timeout e l'intervallo). **Timeout:** quanto attendere la risposta.

La risposta viene verificata - CRC / LRC, slave, funzione, lunghezza - e le eccezioni Modbus sono nominate (per esempio *exception 2: Illegal data address*). I valori letti sono mostrati nella tabella (**Indirizzo**, **Valore**, **Grezzo**) come scelto in **Mostra i registri come:** senza segno / con segno 16 bit, esadecimale, binario, o 32 bit senza segno, con segno e float con la parola alta o bassa prima. Il registro conserva ogni richiesta (>) e risposta (<) con l'orario e il tempo di andata e ritorno, e i contatori mostrano richieste, risposte, eccezioni, trame errate e timeout. Il master Modbus richiede la funzione di licenza *Invio*.

16. Il Plotter seriale



Il Plotter seriale con tre valori

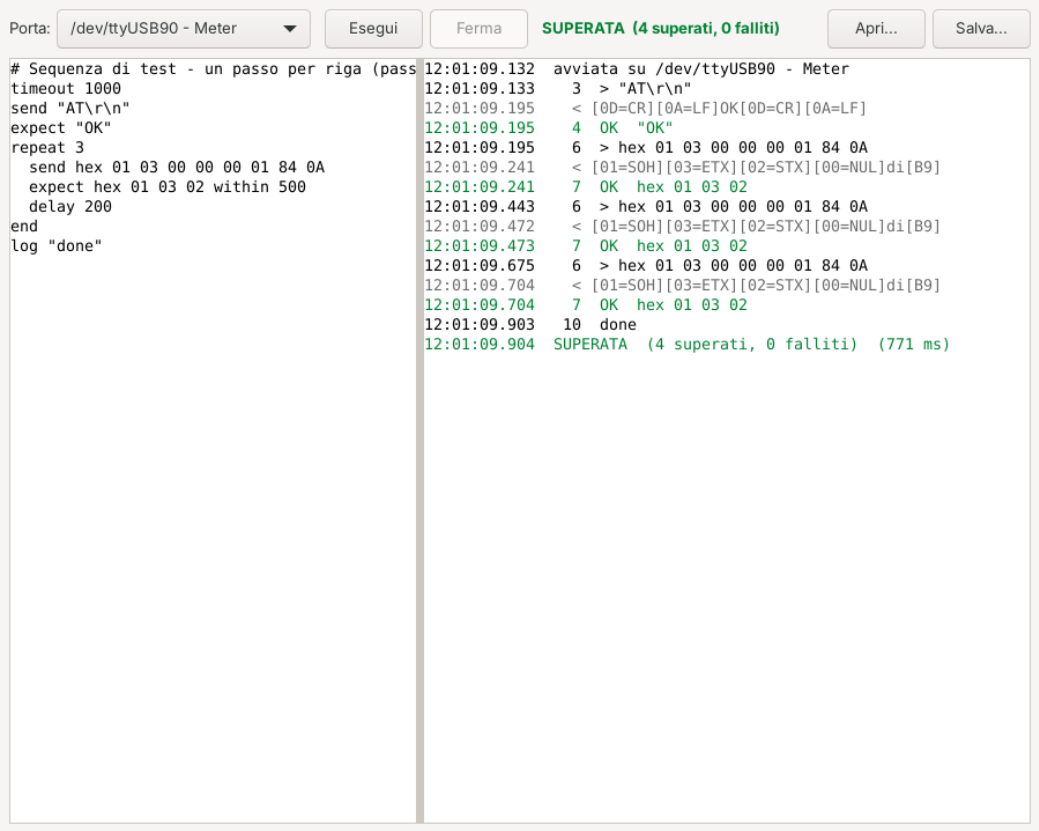
Strumenti > Plotter seriale... (Ctrl-L) disegna i numeri stampati da un dispositivo, come il plotter dell'IDE Arduino. Ogni riga di testo (terminata da LF, CR o CR LF) è un campione; i suoi valori sono separati da virgole, punti e virgola, tabulazioni o spazi, semplici (`21.5,40`) o con nome (`temp:21.5 hum=40`); le parole che non sono numeri vengono saltate (`T= 21.5 C` dà 21.5 di nome `T`). I valori senza nome si chiamano 1, 2, 3...

- **Porta:** la scheda da ascoltare (la sua console continua a funzionare come sempre).
- **Pausa** blocca il disegno; i campioni continuano a essere raccolti.
- **Campioni mostrati:** gli ultimi N campioni (10 - 100000).
- **Scala automatica**, o il fondo e la cima di una scala fissa.
- **Pulisci**, **Esporta CSV...** (campione, orario e ogni valore).

Un colore per valore (fino a 16), una legenda con gli ultimi valori in alto, la scala a sinistra e i numeri del primo e dell'ultimo campione sotto il grafico.

17. Sequenze di test

Copyright © 2026 Fation Coga. Tutti i diritti riservati.



Una sequenza di test superata

Strumenti > Sequenze di test... (Ctrl-T) esegue uno script di passi verso un dispositivo attraverso la porta di una scheda; ogni attesa è superata o fallita. Un passo per riga (passando sopra lo script compare questo elenco):

Passo	Significato
`send "AT\r\n"`	invia testo; sequenze `\r \n \t \0 \\ \"` e `\xHH` per qualsiasi byte
`send hex 01 03 00 00`	invia byte esadecimali
`send mixed AT//0x0D`	invia byte nella notazione Mista
`expect "OK"`	attende che questi byte arrivino dopo l'ultimo invio: superato, o fallito dopo il timeout
`expect hex 01 03 within 200`	... attendendo al massimo 200 ms
`timeout 1000`	l'attesa predefinita di expect (ms; 1000 all'inizio)
`delay 250`	attende 250 ms
`repeat 10` ... `end`	ripete i passi in mezzo (0 = fino all'arresto); si possono annidare
`log "testo"`	scrive il testo nel registro
`onfail continue` / `onfail stop`	dopo un expect fallito: prosegue, o termina l'esecuzione (predefinito)
`# ...`	un commento

Esegui controlla lo script (gli errori sono elencati con la loro riga) e lo esegue; **Ferma** lo termina. Il registro mostra ogni passo con il numero di riga, ciò che è stato inviato (>), ciò che è stato ricevuto (<), **OK** in verde e **FALLITO** in rosso con ciò che era arrivato. Il risultato è **SUPERATA (n superati, 0 falliti)** o **FALLITA (...)**. Lo script è conservato tra le sessioni; **Apri...** e **Salva...** lo leggono e lo scrivono come file (`*.spmseq`). Le sequenze di test richiedono la funzione di licenza *Invio*.

18. Profili

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Un profilo contiene la configurazione delle porte: le impostazioni della porta di ogni scheda, le linee di controllo, l'automazione, la casella di invio, le opzioni di ricezione e della console, i comandi salvati e le regole di evidenziazione - non la lingua, il tema, il livello di log, i limiti del database eventi o la licenza. **File > Esporta profilo...** lo salva in un file (`*.spmprofile`); **File > Importa profilo...** sostituisce la configurazione attuale con quella di un file (dopo una domanda; le porte aperte vengono chiuse) e ricrea le schede. Serve per spostare una configurazione su un altro PC o per tenerne una per progetto.

19. Licenze

SerialPortManager è concesso in licenza per PC. Questo capitolo descrive le licenze dal punto di vista dell'utente.

19.1 Prova, con licenza e senza licenza

Stato	Cosa funziona
Prova (14 giorni dal primo avvio sul PC)	tutte le funzioni
Con licenza	le funzioni della licenza
Senza licenza (prova terminata, nessuna licenza valida)	le porte si aprono, tutto ciò che viene ricevuto è mostrato e registrato; Analizzatore di byte e finestra Protocolli

Le funzioni che una licenza può comprendere:

Funzione	Sblocca
Invio	l'invio dalla casella Invia, comandi salvati, riproduzione, master Modbus, sequenze di test
Risposte automatiche	le risposte automatiche ai messaggi ricevuti
Messaggi periodici	i messaggi periodici
Trasferimento file	Invia file e Cattura file
Inoltro a un'altra porta	Inoltra ricevuti a

Le licenze vengono emesse in edizioni: **professional** comprende tutte le funzioni, anche quelle delle versioni future; **standard** comprende l'invio; si possono emettere anche altre combinazioni.

Usando una funzione non disponibile compare la domanda *"Invio richiede una licenza. ... Aprire la finestra Licenza?"*, con il motivo (per esempio *La tua licenza (edizione: standard) non la include.* oppure *Il periodo di prova è terminato.*).

19.2 La finestra Licenza

Aiuto > Licenza... (Ctrl-K) apre la finestra **Licenza**:

- Un LED colorato e lo stato: **Con licenza** (verde; giallo negli ultimi 14 giorni prima della scadenza), **Versione di prova - N giorno/i rimasti** (blu) o **Senza licenza** (rosso), con il motivo quando una licenza è presente ma rifiutata.
- I dettagli della licenza: **Licenziatario:**, **E-mail:**, **Edizione:**, **ID licenza:**, **Valida su:** (**questo PC**, **N PC, incluso questo**, oppure **qualsiasi PC (licenza di sito)**), **Emessa:**, **Scadenza:** (una data con i giorni rimasti, oppure **mai (perpetua)**), **Aggiornamenti fino al:** (una data, oppure **tutte le versioni**), **Note:**, **File:**.
- **Funzioni:** ogni funzione, **disponibile** o **non inclusa**.
- **ID univoco di questo PC (UID):** l'identificatore di questo PC, con **Copia**.
- I pulsanti **Salva richiesta...**, **Carica licenza...**, **Incolla licenza**, **Rimuovi licenza**, **Apri cartella** e **Chiudi**.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

19.3 Ottenere una licenza

1. Aprire **Aiuto > Licenza...**.
2. Inviare all'emittente della licenza l'UID (fare clic su **Copia** e incollarlo in un'e-mail) oppure il file di richiesta: **Salva richiesta...** lo salva dove si vuole (per esempio per allegarlo a un'e-mail); una copia, license-request.txt, si trova sempre nella cartella dati (**Apri cartella**). La richiesta contiene il prodotto, l'UID, il nome del computer, la versione e - per un rinnovo - l'ID della licenza attuale.
3. Si riceve un file di licenza (per esempio license-SERIALPM-....lic).
4. Installarlo con **Carica licenza...** (scegliendo il file), **Incolla licenza** (se la licenza è arrivata come testo, copiarla e fare clic), oppure trascinando il file sulla finestra Licenza. In alternativa lo si può copiare come license.lic nella cartella dati (o accanto all'exe) prima di avviare il programma.

Il programma controlla subito la licenza: **Licenza installata - licenziatario: NOME.**, oppure **Questa licenza non può essere usata su questo PC:** con il motivo. Le funzioni diventano subito disponibili (risposte automatiche e messaggi periodici partono se i loro interruttori sono attivi).

Rimuovi licenza chiede **Rimuovere la licenza installata? Viene conservata come license.lic.bak nella cartella dati.**

19.4 Che cosa controlla una licenza

- **La firma**: una licenza è firmata digitalmente dall'emittente. Qualsiasi modifica al file la rende non valida (**La licenza non è stata emessa dal titolare della licenza, oppure è stata modificata dopo l'emissione.**).
- **Il prodotto**: una licenza per un altro prodotto viene rifiutata.
- **Il PC**: l'UID è ricavato dall'installazione di Windows del PC. Reinstallare Windows lo cambia (serve una nuova licenza); cambiare l'hardware no. Una licenza può indicare più PC, oppure qualsiasi PC (licenza di sito).
- **Scadenza**: una licenza a tempo funziona fino alla data di scadenza; la barra di stato avvisa negli ultimi 14 giorni.
- **Aggiornamenti fino al**: la licenza copre le versioni rilasciate fino a quella data; una versione più recente richiede un rinnovo (**La licenza copre le versioni rilasciate fino al ...; questa versione è stata rilasciata il .... Continua a usare una versione precedente o rinnova la licenza.**).
- **Revoca**: una licenza revocata viene rifiutata dalle versioni successive.
- **L'orologio**: se la data del PC viene riportata indietro di più di 2 giorni rispetto a una data già vista, la prova e le licenze a tempo vengono sospese finché la data non è corretta. Le licenze perpetue non sono mai interessate.
- **Lo stato della prova** è conservato due volte, con valori di controllo; modificarlo fa terminare la prova (**Le informazioni sulla prova sono state modificate - la prova è terminata.**).

Lo stato della licenza viene ricontrollato una volta al giorno mentre il programma è in esecuzione.

20. Impostazioni

20.1 Tema

Impostazioni > Tema... offre **Chiaro** e **Scuro** (chiaro è il predefinito). Dopo la scelta il programma chiede come applicarla:

- **Riavvia ora**: salva e riavvia automaticamente il programma;

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

- **Usa ora**: salva e usa subito; le parti disegnate dal sistema (barre del titolo, barre di scorrimento) cambiano al prossimo avvio;
- **Salva per il prossimo avvio**: si continua a lavorare, si applica al prossimo avvio;
- **Annulla**: scarta la scelta.

Nel tema scuro i colori di stato sono scelti per restare leggibili (per esempio il verde e il rosso negli analizzatori e nella forma d'onda).

20.2 Lingua

Impostazioni > Lingua... elenca **English** e le lingue il cui file è nella cartella dati (**Italiano**). Segue la stessa domanda del tema. Con **Usa ora** i menu e le finestre aperte da quel momento usano la nuova lingua; le altre etichette della finestra principale cambiano al prossimo avvio. Se un file di lingua non si può caricare, resta la lingua attuale.

20.3 Livello log

Impostazioni > Livello log... - **Scrivi nel log i messaggi fino a:** None, Error, Warning, Info (predefinito), Debug, Verbose, Trace (i nomi dei livelli sono in inglese). I livelli più alti scrivono più dettagli nel Log dell'applicazione e in log.txt; usare Debug o superiore solo per indagare su un problema.

20.4 Apri cartella dati

Impostazioni > Apri cartella dati apre la cartella dati (capitolo 2.3) in Esplora risorse.

20.5 Database eventi

Impostazioni > Database eventi... mostra il file events.db (capitolo 2.3), il numero di eventi e la sua dimensione, e imposta:

- **Registra il traffico seriale (ogni messaggio ricevuto e inviato)** - attivo per impostazione predefinita; disattivato, si conservano solo gli eventi del programma (avvio, porte, licenza, file);
- **Conserva gli eventi per (giorni)** - predefinito 30, **0 = nessun limite**: gli eventi più vecchi vengono eliminati;
- **Dimensione massima (MB)** - predefinito 200, **0 = nessun limite**: gli eventi più vecchi vengono eliminati finché il database ci sta.

I limiti si applicano all'avvio, ogni 10 minuti e quando la finestra si chiude con **OK**. **Esporta in CSV...** salva ogni evento (orario, tipo, testo); **Compatta ora** restituisce al disco lo spazio degli eventi eliminati; **Elimina tutto...** svuota il database dopo una domanda.

21. Menu Aiuto

- **Manuale utente** (F1): questo manuale in PDF, nella lingua in uso.
- **Guida rapida**: la guida breve in PDF.
- **Licenza...** (Ctrl-K): capitolo 19.
- **Informazioni versione...**: la finestra **Informazioni versione** con versione, numero e data della build, tipo di build (**Sviluppo (Debug)**, **Release candidate** o **Versione finale**), sorgente (git), cartella dati e la cronologia delle versioni (dalla più recente).
- **Informazioni su...**: una finestra a schede - **Informazioni** (nome, versione e data di compilazione, descrizione, autore, contatto, pagina del progetto, copyright, licenza e il suo stato, che cosa fa il programma), **Sviluppatore** (e-mail, pagine GitHub, **Segnala un problema via e-mail...**, che apre

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

un'e-mail con i dettagli di questo PC già compilati), **Dipendenze** (ogni libreria con versione, licenza e uso; nomi e licenze sono collegamenti), **Porte** (le porte seriali di questo PC, che cos'è ogni dispositivo, il suo id hardware e la scheda che la usa), **Licenza** (stato, funzioni, UID di questo PC, **Finestra Licenza...**), **Modifiche** (la cronologia delle versioni) e **Sistema** (versioni, cartelle, database, porte, licenza, UID - **Copia negli appunti** per una richiesta di assistenza).

22. Risoluzione dei problemi

Problema	Cosa fare
Impossibile aprire COM3: ...	La porta è usata da un altro programma (chiuderlo), è scollegata, oppure il nome è errato. Fare clic su **Aggiorna** nelle impostazioni della porta e scegliere di nuovo.
La porta non è nell'elenco	Collegare l'adattatore, installarne il driver, fare clic su **Aggiorna**. Le porte virtuali e Bluetooth compaiono solo con il driver attivo.
Quante porte seriali controllare non si può aumentare	Il massimo è il numero di porte presenti ora (al massimo 32). Collegare prima gli adattatori.
Porta già in uso	La porta appartiene a un'altra scheda: scegliere **Scollega e usa qui** oppure un'altra porta.
Non si riceve nulla	Controllare velocità, bit di dati, parità e bit di stop sui due lati; incrociare TX e RX (il TX di un lato sull'RX dell'altro); collegare GND; con **Hardware (RTS/CTS)** verificare che l'altra parte piloti CTS, oppure scegliere **Nessuno**. Con RS-485 scambiare A e B se non arriva nulla.
Il testo ricevuto è illeggibile	Velocità o parità errate, oppure livelli errati (RS-232 direttamente su un ingresso TTL). La pagina Segnale mostra come dovrebbero apparire i byte sulla linea.
Messaggi divisi o uniti	Aumentare o diminuire il **Timeout tra byte (ms)** (capitolo 6), o dividerli per delimitatore, lunghezza o temporizzazione Modbus nelle opzioni di **Ricezione** (8.4).
Scrittura non riuscita (...)	Il timeout di scrittura è scaduto: controllo di flusso che blocca, adattatore rimosso, oppure timeout di scrittura troppo breve.
La porta ha smesso di funzionare	Un adattatore USB è stato scollegato: la porta viene riaperta quando ritorna (6.3), oppure chiuderla e riaprirla.
Invia è disattivato	Aprire la porta; attendere la fine di un trasferimento file.
Troppi messaggi in attesa	Messaggi periodici più veloci di quanto la linea possa trasportare: allungare il periodo o aumentare la velocità.
Le risposte automatiche non rispondono	L'interruttore **Risposte automatiche** è attivo, la regola spuntata, la licenza comprende le Risposte automatiche? Lo schema corrisponde esattamente ai byte ricevuti (controllare la notazione)? Le risposte automatiche si sospendono durante l'invio di un file.
Un messaggio periodico si è fermato	Il suo testo non si può costruire: modificarlo e correggerlo.
L'invio di un file fallisce	Leggere il motivo nella console (per esempio *no confirmation for chunk 4 (expected: ...)*): controllare lo schema di conferma, il timeout di risposta, la velocità e che i due lati usino gli stessi comandi e la stessa dimensione di blocco / gruppo.
Cattura file rifiuta le opzioni	Scegliere un file di uscita e un marcatore di fine oppure un tempo di silenzio.
Il manuale non è stato trovato	Mancano i manuali PDF nella cartella **manuals** accanto all'exe: copiarli lì.
Una licenza viene rifiutata	Leggere il motivo nella finestra Licenza: altro PC (inviare l'UID di questo PC), scaduta, aggiornamenti non coperti (usare una versione precedente o rinnovare), orologio riportato indietro (correggere la data).
La prova è terminata prima del previsto	La data del PC è stata riportata indietro, oppure le informazioni sulla prova sono state modificate.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Problema	Cosa fare
Una linea di controllo non cambia	Le porte virtuali e alcune USB non hanno linee modem (la nota della console è rossa); con il controllo di flusso hardware RTS appartiene al driver.
Il master Modbus non riceve risposta	Controllare indirizzo dello slave, modalità (RTU / ASCII), velocità e parità; con RS-485 l'adattatore deve cambiare direzione da solo. Un timeout con alcuni byte ricevuti indica un formato errato o un timeout troppo breve.
Il plotter non mostra nulla	Il dispositivo deve terminare le righe (LF o CR) e stampare numeri; la riga di stato conta le righe ricevute.
Una sequenza di test fallisce subito	**expect** guarda solo ciò che arriva dopo l'ultimo **send**; allungare il timeout (`within`, `timeout`).

Se un problema persiste, impostare **Impostazioni > Livello log...** su Debug, ripetere l'operazione e inviare all'assistenza log.txt (e, se utile, events.db) dalla cartella dati.

23. Scorciatoie da tastiera e mouse

Tasti / mouse	Dove	Azione
Alt-X	finestra principale	Esci
Ctrl-B	finestra principale	Analizzatore di byte sui byte della casella di invio
Ctrl-P	finestra principale	finestra Protocolli
F1	finestra principale	Manuale utente
Ctrl-K	finestra principale	finestra Licenza
Ctrl-M	finestra principale	Master Modbus
Ctrl-L	finestra principale	Plotter seriale
Ctrl-T	finestra principale	Sequenze di test
Invio	casella Invia	invia
Su / Giù	casella Invia	messaggi inviati prima
Ctrl-F	console	barra di ricerca
F3 / Maiusc-F3	console	corrispondenza successiva / precedente
Invio / Maiusc-Invio, Esc	barra di ricerca	successiva / precedente, chiudi
clic destro	console	menu di analisi
tasto menu contestuale	console	menu di analisi della selezione
doppio clic	elenchi di risposte automatiche, messaggi periodici, valori incrementali	modifica l'elemento
doppio clic	tabella Inserisci carattere	aggiunge il carattere e chiude
clic destro	tabella Inserisci carattere	aggiunge il carattere e tiene aperta la tabella
clic destro	elenchi dell'Analizzatore di byte	copia le righe
rotellina	forma d'onda	scorre in orizzontale
Maiusc + rotellina	forma d'onda	scorre in su / in giù
Ctrl + rotellina	forma d'onda	zoom attorno al mouse
Esc	finestre di dialogo	chiude / annulla

24. Glossario

Termine	Significato
8N1	8 bit di dati, nessuna parità, 1 bit di stop: 10 tempi di bit per byte

Copyright © 2026 Fation Coga. Tutti i diritti riservati.

Termine	Significato
ACK / NACK	conferma / mancata conferma
ASCII	il codice dei caratteri a 7 bit; i caratteri stampabili sono 20-7E esadecimale
Baud rate	simboli (qui bit) al secondo sulla linea
BCC	Block Check Character, lo XOR dei byte
BCD	Binary Coded Decimal: due cifre decimali per byte
Big-endian / little-endian	prima il byte più / meno significativo
Bit di stuffing	un bit di valore opposto inserito dopo cinque bit uguali (CAN)
Blocco / gruppo	la porzione di file inviata / confermata alla volta
Break	la linea TX tenuta a 0 più a lungo di un carattere: un reset o un marcatore di trama
Cattura	il traffico di una porta scritto in un file mentre avviene
Checksum	un valore calcolato dai byte per rilevare errori
Controllo di flusso	il modo in cui il ricevitore mette in pausa il mittente (XON/XOFF o RTS/CTS)
CPOL / CPHA	polarità e fase del clock SPI
CRC	Cyclic Redundancy Check, un checksum robusto
CTS / DSR / DCD / RI	ingressi modem: pronto a trasmettere, apparato pronto, portante rilevata, indicatore di chiamata
Dominante / recessivo	livelli dei bus CAN e LIN: il dominante (0) prevale sul recessivo (1)
Interrogazione (poll)	una richiesta inviata di continuo a intervalli
Licenza di sito	una licenza valida su qualsiasi PC del licenziatario
LRC	Longitudinal Redundancy Check: complemento a due della somma
Marcatore di fine	byte che segnano la fine di un file
Modalità burst	invio di file che attende una conferma dopo ogni blocco
Parametri Rocksoft	Width, Poly, Init, RefIn, RefOut, XorOut: la descrizione pubblicata di un CRC
Parità	un bit in più che rende dispari o pari il numero di uni
Prefisso	byte inviati prima di un file per annunciarlo
Profilo	l'intera configurazione delle porte in un file
Recupero	continuazione di un trasferimento interrotto da dove il ricevitore si è fermato
RTS / DTR	uscite modem: richiesta di trasmissione, terminale pronto
RX / TX	ricevuto / trasmesso
Slot	una scheda della finestra principale, con la propria porta
Suddivisione (framing)	come un flusso di byte viene diviso in messaggi (pausa, delimitatore, lunghezza...)
Timeout tra byte	il silenzio che chiude un messaggio ricevuto
UART	l'interfaccia seriale asincrona (start, dati, parità, stop)
UID	l'identificatore univoco di un PC usato dalla licenza

25. Note legali

Copyright (C) 2026 Fation Coga. Tutti i diritti riservati.

SerialPortManager e questo manuale sono di proprietà di Fation Coga. SerialPortManager è un software proprietario, concesso in licenza per PC secondo i termini del suo Contratto di licenza (LICENSE.md): è concesso in licenza, non venduto. Nessuna parte di questo manuale può essere riprodotta, distribuita o tradotta, in qualsiasi forma o con qualsiasi mezzo, senza il permesso scritto del titolare dei diritti.

Il software è fornito "così com'è", senza garanzie di alcun tipo, nei limiti consentiti dalla legge; vedere il Contratto di licenza per l'esclusione delle garanzie e la limitazione di responsabilità. SerialPortManager comunica con apparecchiature esterne: l'utente è responsabile di verificarne l'idoneità e del funzionamento sicuro di qualsiasi apparecchiatura collegata.

Componenti di terze parti. SerialPortManager include o collega i seguenti componenti, ognuno con la propria licenza; le relative note sono nel file THIRD-PARTY-NOTICES.md fornito accanto all'exe:

- wxWidgets - wxWindows Library Licence 3.1, di Julian Smart, Vadim Zeitlin, Stefan Csomor, Robert Roebling e altri membri del team wxWidgets (https://www.wxwidgets.org/);
- SQLite - pubblico dominio (https://www.sqlite.org/);
- serial - il fork CogaF di wjwwood/serial, licenza MIT, Copyright (c) 2012 William Woodall, John Harrison (https://github.com/wjwwood/serial);
- TweetNaCl - pubblico dominio, di Daniel J. Bernstein, Wesley Janssen, Tanja Lange, Peter Schwabe, Matthew Dempsky e Sjaak Smetsers, usato per l'aritmetica delle firme Ed25519 (https://tweetnacl.cr.yp.to/).

Marchi. Microsoft e Windows sono marchi di Microsoft Corporation. wxWidgets è il nome del progetto wxWidgets, del team wxWidgets. SQLite è un marchio registrato di Hipp, Wyrick & Company, Inc. Modbus è un marchio registrato di Schneider Electric USA, Inc. I²C è un marchio di NXP B.V. 1-Wire e iButton sono marchi registrati di Maxim Integrated Products, Inc., ora parte di Analog Devices, Inc. CAN è standardizzato nella norma ISO 11898 ed è stato sviluppato da Robert Bosch GmbH. LIN è standardizzato nella norma ISO 17987. NMEA 0183 è uno standard della National Marine Electronics Association. JTAG è normato dalla IEEE 1149.1. Arm e CoreSight sono marchi di Arm Limited. Tutti gli altri nomi e marchi appartengono ai rispettivi titolari e sono usati solo per identificare i prodotti e i protocolli; il loro uso non implica alcuna affiliazione con essi né alcuna loro approvazione.

Copyright © 2026 Fation Coga. Tutti i diritti riservati.